

PYTHON INTERVIEW QUESTIONS

1. What is Python?

Answer: Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum in 1991. It emphasizes code readability with significant use of indentation.

Example:

```
# Simple Python program
```

```
print("Hello, World!")
```

```
name = "Krishna"
```

```
age = 25
```

```
print(f"My name is {name} and I am {age} years old.")
```

2. What are the key features of Python?

Answer:

- Easy to learn and read (simple syntax)
- Interpreted language (no compilation needed)
- Dynamically typed (no need to declare variable types)
- Object-Oriented and Functional programming support
- Extensive standard library
- Platform independent (runs on Windows, Mac, Linux)
- Open source and free
- Large community support

Example:

```
# Dynamic typing
```

```

x = 10      # int
x = "hello" # now it's a string
x = [1, 2, 3] # now it's a list

# Multiple paradigms
# Functional
result = list(map(lambda x: x*2, [1, 2, 3]))

# OOP
class Dog:
    def __init__(self, name):
        self.name = name

```

3. What are Python data types?

Answer: Python has the following built-in data types:

- Numeric: int, float, complex
- Sequence: str, list, tuple, range
- Set: set, frozenset
- Mapping: dict
- Boolean: bool
- Binary: bytes, bytearray, memoryview
- None: NoneType

Example:

```

# Numeric
a = 10      # int
b = 3.14    # float
c = 2 + 3j  # complex

```

```

# Sequence
s = "hello"    # str
lst = [1, 2, 3] # list
tup = (1, 2, 3) # tuple

# Set
st = {1, 2, 3} # set

# Mapping
d = {"name": "Krishna", "age": 25} # dict

# Boolean
flag = True    # bool

# None
x = None       # NoneType

print(type(a)) # <class 'int'>

```

4. Difference between List and Tuple?

Answer:

Feature	List	Tuple	
-----	-----	-----	
Syntax	[]	()	
Mutable	Yes	No	
Speed	Slower	Faster	
Memory	More	Less	

Methods	Many (append etc)	Few (count, index)
Use case	Dynamic data	Fixed data

Example:

```
# List - mutable
my_list = [1, 2, 3]
my_list[0] = 10    # Allowed
my_list.append(4)   # Allowed
print(my_list)     # [10, 2, 3, 4]

# Tuple - immutable
my_tuple = (1, 2, 3)
# my_tuple[0] = 10  # TypeError! Not allowed
print(my_tuple[0]) # 1
```

5. Difference between List and Set?

Answer:

Feature	List	Set
Syntax	[]	{ }
Ordered	Yes	No
Duplicates	Allowed	Not allowed
Indexing	Supported	Not supported
Mutable	Yes	Yes

Example:

```
# List allows duplicates and maintains order
my_list = [1, 2, 2, 3, 3]
```

```

print(my_list)    # [1, 2, 2, 3, 3]
print(my_list[0]) # 1

# Set removes duplicates, no order
my_set = {1, 2, 2, 3, 3}
print(my_set)     # {1, 2, 3}
# print(my_set[0]) # TypeError! No indexing

```

6. Difference between List and Dictionary?

Answer:

Feature	List	Dictionary	
-----	-----	-----	
Syntax	[]	{}	
Access	By index	By key	
Structure	Sequence of items	Key-value pairs	
Ordered	Yes	Yes (Python 3.7+)	
Duplicates	Allowed	Keys must be unique	

Example:

```

# List - accessed by index
fruits = ["apple", "banana", "cherry"]
print(fruits[0]) # apple

# Dictionary - accessed by key
person = {"name": "Krishna", "age": 25, "city": "Hyderabad"}
print(person["name"]) # Krishna

```

7. What is a Dictionary in Python?

Answer: A dictionary is an unordered (ordered from 3.7+) collection of key-value pairs. Keys must be unique and immutable. Values can be any type.

Example:

```
# Creating dictionary
```

```
student = {"name": "Krishna", "age": 25, "marks": [90, 85, 88]}
```

```
# Accessing
```

```
print(student["name"])    # Krishna
```

```
print(student.get("age")) # 25
```

```
# Adding/Updating
```

```
student["city"] = "Hyderabad" # add new key
```

```
student["age"] = 26           # update existing
```

```
# Methods
```

```
print(student.keys())       # dict_keys(['name', 'age', 'marks', 'city'])
```

```
print(student.values())     # dict_values([90, 85, 88, 'Hyderabad'])
```

```
print(student.items())     # dict_items([('name', 'Krishna'), ('age', 25), ('marks', [90, 85, 88]), ('city', 'Hyderabad')])
```

```
# Deleting
```

```
del student["marks"]
```

```
student.pop("city")
```

8. What is a Tuple in Python?

Answer: A tuple is an ordered, immutable collection of elements. Once created, elements cannot be added, removed, or changed. Tuples use parentheses ().

Example:

```
# Creating tuple
colors = ("red", "green", "blue")
single = (5,) # single element tuple needs comma
```

```
# Accessing
print(colors[0]) # red
print(colors[-1]) # blue
print(colors[0:2]) # ('red', 'green')
```

```
# Tuple methods
print(colors.count("red")) # 1
print(colors.index("green")) # 1
```

```
# Unpacking
r, g, b = colors
print(r) # red
```

```
# Tuple is immutable
# colors[0] = "yellow" # TypeError!
```

9. What is a Set in Python?

Answer: A set is an unordered collection of unique elements. It does not allow duplicates and does not support indexing. Sets are useful for mathematical operations like union, intersection.

Example:

```
# Creating set
```

```

nums = {1, 2, 3, 4, 5}
nums2 = {4, 5, 6, 7, 8}

# Adding/Removing
nums.add(6)
nums.remove(1)    # raises error if not found
nums.discard(100) # no error if not found

# Set operations
print(nums | nums2) # Union
print(nums & nums2) # Intersection
print(nums - nums2) # Difference
print(nums ^ nums2) # Symmetric difference

# Remove duplicates from list
my_list = [1, 1, 2, 2, 3]
unique = list(set(my_list))
print(unique) # [1, 2, 3]

```

10. What is String in Python?

Answer: A string is an immutable sequence of characters enclosed in single, double, or triple quotes. Strings support indexing, slicing, and many built-in methods.

Example:

```

# Creating strings
s1 = 'Hello'
s2 = "World"
s3 = """Multi
line string"""

```



```
# String operations
```

```
print(s1 + " " + s2) # Hello World
print(s1 * 3)        # HelloHelloHello
print(s1[0])         # H
print(s1[1:4])        # ell
print(s1[::-1])       # olleH
```

```
# String methods
```

```
text = "hello world"
print(text.upper())   # HELLO WORLD
print(text.title())   # Hello World
print(text.split())   # ['hello', 'world']
print(text.replace("hello", "hi")) # hi world
print(text.find("world")) # 6
print(len(text))      # 11
```

```
# f-string formatting
```

```
name = "Krishna"
print(f"Hello {name}!") # Hello Krishna!
```

11. What are mutable and immutable data types?

Answer:

- Mutable: Can be changed after creation. Examples: list, dict, set
- Immutable: Cannot be changed after creation. Examples: int, float, str, tuple, frozenset

Example:

```
# Mutable - list
lst = [1, 2, 3]
```

```
lst[0] = 100
print(lst)    # [100, 2, 3] - changed in place
print(id(lst)) # same id
```

```
# Immutable - string
s = "hello"
# s[0] = "H"    # TypeError!
s = "Hello"     # creates new object
print(s)        # Hello (new object, different id)
```

```
# Immutable - tuple
t = (1, 2, 3)
# t[0] = 10     # TypeError!
```

```
# Mutable - dictionary
d = {"a": 1}
d["b"] = 2      # Allowed
print(d)        # {'a': 1, 'b': 2}
```

12. What is List Comprehension?

Answer: List comprehension is a concise way to create lists in a single line. It consists of an expression followed by a for clause and optional if conditions.

Syntax: [expression for item in iterable if condition]

Example:

```
# Basic list comprehension
squares = [x**2 for x in range(1, 6)]
print(squares) # [1, 4, 9, 16, 25]
```

```
# With condition
```

```
evens = [x for x in range(10) if x % 2 == 0]
```

```
print(evens)    # [0, 2, 4, 6, 8]
```

```
# String manipulation
```

```
words = ["hello", "world", "python"]
```

```
upper_words = [w.upper() for w in words]
```

```
print(upper_words) # ['HELLO', 'WORLD', 'PYTHON']
```

```
# Nested list comprehension
```

```
matrix = [[i*j for j in range(1,4)] for i in range(1,4)]
```

```
print(matrix)    # [[1,2,3], [2,4,6], [3,6,9]]
```

```
# With if-else
```

```
result = ["even" if x%2==0 else "odd" for x in range(5)]
```

```
print(result)    # ['even', 'odd', 'even', 'odd', 'even']
```

13. What is Dictionary Comprehension?

Answer: Dictionary comprehension is a concise way to create dictionaries in a single line using {key: value for item in iterable}.

Example:

```
# Basic dict comprehension
```

```
squares = {x: x**2 for x in range(1, 6)}
```

```
print(squares)   # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

```
# From two lists
```

```
keys = ["name", "age", "city"]
```

```
values = ["Krishna", 25, "Hyderabad"]
person = {k: v for k, v in zip(keys, values)}
print(person)    # {'name': 'Krishna', 'age': 25, 'city': 'Hyderabad'}
```

```
# With condition
```

```
even_squares = {x: x**2 for x in range(10) if x % 2 == 0}
print(even_squares) # {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

```
# Swapping keys and values
```

```
original = {"a": 1, "b": 2, "c": 3}
swapped = {v: k for k, v in original.items()}
print(swapped)    # {1: 'a', 2: 'b', 3: 'c'}
```

14. What is Lambda Function?

Answer: A lambda function is an anonymous (nameless) small function defined using the lambda keyword. It can have any number of arguments but only one expression.

Syntax: lambda arguments: expression

Example:

```
# Basic lambda
```

```
add = lambda a, b: a + b
print(add(3, 5))    # 8
```

```
# Lambda with map
```

```
nums = [1, 2, 3, 4, 5]
doubled = list(map(lambda x: x * 2, nums))
print(doubled)      # [2, 4, 6, 8, 10]
```

```
# Lambda with filter
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)      # [2, 4]

# Lambda with sorted
students = [("Krishna", 90), ("Ravi", 85), ("Priya", 95)]
sorted_students = sorted(students, key=lambda x: x[1], reverse=True)
print(sorted_students) # [('Priya', 95), ('Krishna', 90), ('Ravi', 85)]

# Lambda with multiple arguments
power = lambda base, exp: base ** exp
print(power(2, 3))   # 8
```

15. What are *args and **kwargs?

Answer:

- *args: Allows a function to accept any number of positional arguments as a tuple.
- **kwargs: Allows a function to accept any number of keyword arguments as a dictionary.

Example:

```
# *args - variable positional arguments
def add(*args):
    return sum(args)

print(add(1, 2, 3))    # 6
print(add(1, 2, 3, 4, 5)) # 15

# **kwargs - variable keyword arguments
def display_info(**kwargs):
```

```

    for key, value in kwargs.items():
        print(f"{key}: {value}")

display_info(name="Krishna", age=25, city="Hyderabad")
# name: Krishna
# age: 25
# city: Hyderabad

# Using both together
def mixed_func(a, b, *args, **kwargs):
    print(f"a={a}, b={b}")
    print(f"args={args}")
    print(f"kwargs={kwargs}")

mixed_func(1, 2, 3, 4, x=10, y=20)
# a=1, b=2
# args=(3, 4)
# kwargs={'x': 10, 'y': 20}

```

16. What is a Generator?

Answer: A generator is a function that returns an iterator using the yield keyword. It generates values lazily (one at a time), saving memory. Instead of storing all values in memory, it produces them on the fly.

Example:

```

# Generator function
def count_up_to(n):
    i = 1
    while i <= n:

```

```
yield i
```

```
i += 1
```

```
counter = count_up_to(5)
```

```
print(next(counter)) # 1
```

```
print(next(counter)) # 2
```

```
# Iterate through generator
```

```
for num in count_up_to(5):
```

```
    print(num) # 1, 2, 3, 4, 5
```

```
# Generator expression (like list comprehension but with ())
```

```
squares = (x**2 for x in range(1000000))
```

```
print(next(squares)) # 0
```

```
print(next(squares)) # 1
```

```
# Memory efficient - Fibonacci generator
```

```
def fibonacci():
```

```
    a, b = 0, 1
```

```
    while True:
```

```
        yield a
```

```
        a, b = b, a + b
```

```
fib = fibonacci()
```

```
for _ in range(10):
```

```
    print(next(fib), end=" ") # 0 1 1 2 3 5 8 13 21 34
```

17. Difference between Generator and Iterator?

Answer:

Feature	Generator	Iterator	
-----	-----	-----	
Creation	Using yield keyword	Using __iter__ and __next__	
Syntax	Function with yield	Class with iter/next methods	
Memory	Very efficient	Depends on implementation	
Code	Less code	More code (class needed)	
State	Automatically maintained	Manually maintained	

Example:

Iterator - using class

class MyRange:

def __init__(self, start, end):

self.current = start

self.end = end

def __iter__(self):

return self

def __next__(self):

if self.current >= self.end:

raise StopIteration

value = self.current

self.current += 1

return value

for i in MyRange(1, 4):

print(i) # 1, 2, 3

Generator - same thing, much simpler


```
def my_range(start, end):
    current = start
    while current < end:
        yield current
        current += 1

for i in my_range(1, 4):
    print(i) # 1, 2, 3
```

18. What is an Iterator?

Answer: An iterator is an object that implements `__iter__()` and `__next__()` methods. It allows you to traverse through all elements of a collection one at a time. Every generator is an iterator, but not every iterator is a generator.

Example:

```
# Built-in iterators
my_list = [1, 2, 3]
my_iter = iter(my_list) # creates iterator
print(next(my_iter))    # 1
print(next(my_iter))    # 2
print(next(my_iter))    # 3
# print(next(my_iter)) # StopIteration error
```

```
# Custom iterator class
class EvenNumbers:
    def __init__(self, limit):
        self.limit = limit
        self.current = 0
```

```

def __iter__(self):
    return self

def __next__(self):
    self.current += 2
    if self.current > self.limit:
        raise StopIteration
    return self.current

```

```

evens = EvenNumbers(10)
for num in evens:
    print(num) # 2, 4, 6, 8, 10

```

19. What is the yield keyword?

Answer: yield is used inside a function to make it a generator. When the function is called, it doesn't execute immediately. Instead, it returns a generator object. Each time next() is called, the function executes until it hits yield, returns the value, and pauses. It resumes from where it left off on the next call.

Example:

```

# Basic yield
def simple_generator():
    print("First")
    yield 1
    print("Second")
    yield 2
    print("Third")
    yield 3

```

```

gen = simple_generator()
print(next(gen)) # prints "First", returns 1
print(next(gen)) # prints "Second", returns 2
print(next(gen)) # prints "Third", returns 3

```

```

# yield with send()
def accumulator():
    total = 0
    while True:
        value = yield total
        total += value

```

```

acc = accumulator()
next(acc)      # Initialize (returns 0)
print(acc.send(5)) # 5
print(acc.send(10)) # 15
print(acc.send(3)) # 18

```

20. What is the difference between return and yield?

Answer:

Feature	return	yield	
-----	-----	-----	
Function	Ends the function	Pauses the function	
Value	Returns one value	Can yield multiple values	
Memory	Stores all in memory	Generates one at a time	
State	Doesn't save state	Saves state between calls	
Type	Regular function	Generator function	

Example:

return - stores all values in memory

```
def get_squares_return(n):
```

```
    result = []
```

```
    for i in range(n):
```

```
        result.append(i ** 2)
```

```
    return result
```

yield - generates one value at a time

```
def get_squares_yield(n):
```

```
    for i in range(n):
```

```
        yield i ** 2
```

return: all values computed and stored at once

```
squares_list = get_squares_return(5)
```

```
print(squares_list) # [0, 1, 4, 9, 16]
```

yield: values computed lazily

```
squares_gen = get_squares_yield(5)
```

```
print(next(squares_gen)) # 0
```

```
print(next(squares_gen)) # 1
```

Memory comparison for large data

```
import sys
```

```
list_size = sys.getsizeof(get_squares_return(10000))
```

```
gen_size = sys.getsizeof(get_squares_yield(10000))
```

```
print(f"List: {list_size} bytes") # ~87000+ bytes
```

```
print(f"Generator: {gen_size} bytes") # ~120 bytes
```

21. What is Exception Handling?

Answer: Exception handling is a mechanism to handle runtime errors gracefully without crashing the program. Python uses try-except blocks to catch and handle exceptions. It prevents program termination and allows recovery from errors.

Example:

```
# Basic exception handling
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

```
# Handling multiple exceptions
try:
    num = int("abc")
except ValueError:
    print("Invalid number format!")
except TypeError:
    print("Type error occurred!")
```

```
# Catching exception details
try:
    lst = [1, 2, 3]
    print(lst[10])
except IndexError as e:
    print(f"Error: {e}") # Error: list index out of range
```

22. What are try, except, else and finally blocks?

Answer:

- try: Contains code that might raise an exception
- except: Handles the exception if one occurs
- else: Executes if no exception occurred in try block
- finally: Always executes, regardless of exception (used for cleanup)

Example:

```
def divide(a, b):  
    try:  
        result = a / b  
    except ZeroDivisionError:  
        print("Error: Division by zero!")  
        return None  
    else:  
        print("Division successful!")  
        return result  
    finally:  
        print("This always runs (cleanup)")
```

```
print(divide(10, 2))  
# Division successful!  
# This always runs (cleanup)  
# 5.0
```

```
print(divide(10, 0))  
# Error: Division by zero!  
# This always runs (cleanup)  
# None
```

Practical example - file handling

```
try:
```

```
f = open("data.txt", "r")
content = f.read()
except FileNotFoundError:
    print("File not found!")
else:
    print(content)
finally:
    print("Closing resources")
```

23. How do you create custom exceptions?

Answer: Custom exceptions are created by inheriting from the Exception class or any of its subclasses. This allows you to define application-specific errors.

Example:

```
# Simple custom exception
class InvalidAgeError(Exception):
    def __init__(self, age, message="Age must be between 0 and 150"):
        self.age = age
        self.message = message
        super().__init__(self.message)

    def set_age(age):
        if age < 0 or age > 150:
            raise InvalidAgeError(age)
        print(f"Age set to {age}")

try:
    set_age(200)
except InvalidAgeError as e:
```

```

    print(f"Error: {e.message}, got {e.age}")

# Multiple custom exceptions
class InsufficientFundsError(Exception):
    pass

class AccountLockedError(Exception):
    pass

class BankAccount:
    def __init__(self, balance):
        self.balance = balance
        self.locked = False

    def withdraw(self, amount):
        if self.locked:
            raise AccountLockedError("Account is locked!")
        if amount > self.balance:
            raise InsufficientFundsError(f"Need {amount}, have {self.balance}")
        self.balance -= amount

account = BankAccount(100)
try:
    account.withdraw(200)
except InsufficientFundsError as e:
    print(e) # Need 200, have 100

```

24. What is Object-Oriented Programming (OOP)?

Answer: OOP is a programming paradigm that organizes code into objects (instances of classes). Objects contain data (attributes) and behavior (methods). OOP helps in code reusability, modularity, and easier maintenance.

Example:

```
class Car:
    def __init__(self, brand, model, year):
        self.brand = brand
        self.model = model
        self.year = year
        self.speed = 0

    def accelerate(self, amount):
        self.speed += amount
        print(f"{self.brand} {self.model} speed: {self.speed} km/h")

    def brake(self):
        self.speed = max(0, self.speed - 10)
        print(f"Speed after braking: {self.speed} km/h")

    def __str__(self):
        return f"{self.year} {self.brand} {self.model}"

# Creating objects
car1 = Car("Toyota", "Camry", 2023)
car2 = Car("Honda", "Civic", 2022)

car1.accelerate(50) # Toyota Camry speed: 50 km/h
car1.brake()        # Speed after braking: 40 km/h
print(car1)         # 2023 Toyota Camry
```

25. What are the four pillars of OOP?

Answer:

1. Encapsulation - Bundling data and methods together, hiding internal details
2. Abstraction - Hiding complex implementation, showing only necessary features
3. Inheritance - Creating new classes from existing classes (code reuse)
4. Polymorphism - Same method name behaving differently for different classes

Example:

```
from abc import ABC, abstractmethod
```

```
# Abstraction
```

```
class Shape(ABC):
```

```
    @abstractmethod
```

```
    def area(self):
```

```
        pass
```

```
# Encapsulation
```

```
class Circle(Shape):
```

```
    def __init__(self, radius):
```

```
        self.__radius = radius # private attribute
```

```
    def get_radius(self):
```

```
        return self.__radius
```

```
    def area(self):
```

```
        return 3.14 * self.__radius ** 2
```

```
# Inheritance
```

```
class ColoredCircle(Circle):  
    def __init__(self, radius, color):  
        super().__init__(radius)  
        self.color = color
```

Polymorphism

```
class Rectangle(Shape):  
    def __init__(self, width, height):  
        self.width = width  
        self.height = height  
  
    def area(self):  
        return self.width * self.height
```

```
shapes = [Circle(5), Rectangle(4, 6)]  
for shape in shapes:  
    print(f"Area: {shape.area()}") # Same method, different behavior
```

26. What is Encapsulation?

Answer: Encapsulation is the bundling of data (attributes) and methods that operate on that data within a single class, and restricting direct access to some components. Python uses naming conventions:

- public: name (accessible everywhere)
- protected: `_name` (convention, accessible in class and subclasses)
- private: `__name` (name mangling, not easily accessible outside)

Example:

```
class BankAccount:  
    def __init__(self, owner, balance):
```

```
self.owner = owner      # public
self._account_type = "savings" # protected
self.__balance = balance # private
```

```
def deposit(self, amount):
    if amount > 0:
        self.__balance += amount
        print(f"Deposited {amount}. Balance: {self.__balance}")

def withdraw(self, amount):
    if amount <= self.__balance:
        self.__balance -= amount
        print(f"Withdrew {amount}. Balance: {self.__balance}")
    else:
        print("Insufficient funds!")

def get_balance(self):
    return self.__balance
```

```
acc = BankAccount("Krishna", 1000)
acc.deposit(500)          # Deposited 500. Balance: 1500
acc.withdraw(200)         # Withdrew 200. Balance: 1300
print(acc.owner)          # Krishna (public - accessible)
print(acc._account_type)  # savings (protected - accessible but shouldn't)
# print(acc.__balance)    # AttributeError! (private)
print(acc.get_balance())  # 1300 (access through method)
```

27. What is Abstraction?

Answer: Abstraction hides complex implementation details and shows only the necessary features. In Python, abstraction is achieved using abstract classes (ABC module). An abstract class cannot be instantiated and may contain abstract methods that must be implemented by subclasses.

Example:

```
from abc import ABC, abstractmethod

class PaymentProcessor(ABC):
    @abstractmethod
    def process_payment(self, amount):
        pass

    @abstractmethod
    def refund(self, amount):
        pass

    def validate_amount(self, amount): # concrete method
        return amount > 0

class CreditCardPayment(PaymentProcessor):
    def process_payment(self, amount):
        if self.validate_amount(amount):
            print(f"Processing credit card payment of ${amount}")

    def refund(self, amount):
        print(f"Refunding ${amount} to credit card")

class UPIPayment(PaymentProcessor):
    def process_payment(self, amount):
        if self.validate_amount(amount):
```

```

        print(f"Processing UPI payment of ${amount}")

    def refund(self, amount):
        print(f"Refunding ${amount} via UPI")

# payment = PaymentProcessor() # TypeError! Can't instantiate abstract class
cc = CreditCardPayment()
cc.process_payment(100) # Processing credit card payment of $100

upi = UPIPayment()
upi.process_payment(50) # Processing UPI payment of $50

```

28. What is Inheritance?

Answer: Inheritance allows a class (child/derived) to inherit attributes and methods from another class (parent/base). It promotes code reusability.

Types: Single, Multiple, Multilevel, Hierarchical, Hybrid

Example:

```

# Single Inheritance
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass

class Dog(Animal):
    def speak(self):

```

```
    return f"{self.name} says Woof!"
```

```
# Multiple Inheritance
```

```
class Flyable:
```

```
    def fly(self):
```

```
        return "I can fly!"
```

```
class Swimmable:
```

```
    def swim(self):
```

```
        return "I can swim!"
```

```
class Duck(Animal, Flyable, Swimmable):
```

```
    def speak(self):
```

```
        return f"{self.name} says Quack!"
```

```
# Multilevel Inheritance
```

```
class Puppy(Dog):
```

```
    def play(self):
```

```
        return f"{self.name} is playing!"
```

```
# Usage
```

```
dog = Dog("Buddy")
```

```
print(dog.speak())    # Buddy says Woof!
```

```
duck = Duck("Donald")
```

```
print(duck.speak())  # Donald says Quack!
```

```
print(duck.fly())    # I can fly!
```

```
print(duck.swim())   # I can swim!
```

```
puppy = Puppy("Max")
```

```
print(puppy.speak()) # Max says Woof! (inherited from Dog)
print(puppy.play())  # Max is playing!
```

```
# Method Resolution Order
print(Duck.__mro__)
```

29. What is Polymorphism?

Answer: Polymorphism means "many forms". It allows the same method name to behave differently based on the object calling it. In Python, polymorphism is achieved through method overriding and duck typing.

Example:

```
# Method Overriding (Runtime Polymorphism)
```

```
class Shape:
```

```
    def area(self):
```

```
        pass
```

```
    def description(self):
```

```
        return f"I am a {self.__class__.__name__} with area {self.area()}"
```

```
class Circle(Shape):
```

```
    def __init__(self, radius):
```

```
        self.radius = radius
```

```
    def area(self):
```

```
        return 3.14 * self.radius ** 2
```

```
class Square(Shape):
```

```
    def __init__(self, side):
```



```
self.side = side
```

```
def area(self):  
    return self.side ** 2
```

```
class Triangle(Shape):  
    def __init__(self, base, height):  
        self.base = base  
        self.height = height  
  
    def area(self):  
        return 0.5 * self.base * self.height
```

```
# Same method, different behavior  
shapes = [Circle(5), Square(4), Triangle(6, 3)]  
for shape in shapes:  
    print(shape.description())  
# I am a Circle with area 78.5  
# I am a Square with area 16  
# I am a Triangle with area 9.0
```

```
# Built-in polymorphism  
print(len("hello"))    # 5 (string)  
print(len([1, 2, 3]))  # 3 (list)  
print(len({"a": 1}))   # 1 (dict)
```

30. Difference between Method Overloading and Overriding?

Answer:

Feature	Overloading	Overriding	
-----	-----	-----	
Definition	Same method, different params	Redefine parent method	
Classes	Same class	Parent and child class	
Python	Not directly supported	Fully supported	
Binding	Compile time (static)	Runtime (dynamic)	

Note: Python doesn't support traditional method overloading. We use default args or *args.

Example:

```
# Method Overloading (Python way - using default args)
```

```
class Calculator:
```

```
    def add(self, a=0, b=0, c=0):
```

```
        return a + b + c
```

```
calc = Calculator()
```

```
print(calc.add(1, 2))    # 3
```

```
print(calc.add(1, 2, 3)) # 6
```

```
# Using *args for true flexibility
```

```
class FlexCalculator:
```

```
    def add(self, *args):
```

```
        return sum(args)
```

```
fc = FlexCalculator()
```

```
print(fc.add(1, 2))      # 3
```

```
print(fc.add(1, 2, 3, 4)) # 10
```

```
# Method Overriding
```

```
class Parent:
```

```

def greet(self):
    return "Hello from Parent"

class Child(Parent):
    def greet(self): # overrides parent method
        return "Hello from Child"

class GrandChild(Child):
    def greet(self):
        parent_msg = super().greet() # call parent's method
        return f"{parent_msg} -> and GrandChild"

p = Parent()
c = Child()
gc = GrandChild()
print(p.greet()) # Hello from Parent
print(c.greet()) # Hello from Child
print(gc.greet()) # Hello from Child -> and GrandChild

```

31. What is a Class?

Answer: A class is a blueprint/template for creating objects. It defines attributes (data) and methods (functions) that the objects will have. Classes encapsulate data and behavior together.

Example:

```

class Employee:
    # Class variable (shared by all instances)
    company = "TechCorp"

    def __init__(self, name, salary):

```

```

    # Instance variables (unique to each object)

    self.name = name

    self.salary = salary

def display(self):

    print(f"{self.name} works at {self.company}, earns {self.salary}")

def give_raise(self, percent):

    self.salary *= (1 + percent/100)

# Creating objects from class
emp1 = Employee("Krishna", 50000)
emp2 = Employee("Ravi", 60000)

emp1.display() # Krishna works at TechCorp, earns 50000
emp1.give_raise(10)
emp1.display() # Krishna works at TechCorp, earns 55000.0

```

32. What is an Object?

Answer: An object is an instance of a class. It is a concrete entity created from the class blueprint. Each object has its own set of attributes and can use the methods defined in the class.

Example:

```

class Dog:

    def __init__(self, name, breed):

        self.name = name

        self.breed = breed

```

```

def bark(self):
    return f"{self.name} says Woof!"

# Objects (instances)
dog1 = Dog("Buddy", "Labrador")
dog2 = Dog("Max", "German Shepherd")

# Each object has its own data
print(dog1.name) # Buddy
print(dog2.name) # Max
print(dog1.bark()) # Buddy says Woof!

# Check if object is instance of class
print(isinstance(dog1, Dog)) # True
print(type(dog1))          # <class '__main__.Dog'>

# Objects are independent
dog1.name = "Rocky"
print(dog1.name) # Rocky
print(dog2.name) # Max (unchanged)

```

33. What is Constructor (`__init__`)?

Answer: `__init__` is a special method (dunder method) that is automatically called when an object is created. It initializes the object's attributes. It's Python's constructor equivalent.

Example:

```

class Student:
    def __init__(self, name, age, grade=None):

```

```
self.name = name
self.age = age
self.grade = grade or "Not assigned"
print(f"Student {name} created!")
```

```
def __str__(self):
    return f"Student({self.name}, {self.age}, {self.grade})"
```

__init__ is called automatically

```
s1 = Student("Krishna", 20, "A") # Student Krishna created!
s2 = Student("Ravi", 21)         # Student Ravi created!
print(s1) # Student(Krishna, 20, A)
print(s2) # Student(Ravi, 21, Not assigned)
```

__init__ with validation

```
class PositiveNumber:
```

```
    def __init__(self, value):
        if value < 0:
            raise ValueError("Value must be positive!")
        self.value = value
```

```
num = PositiveNumber(5) # Works
```

```
# num = PositiveNumber(-1) # ValueError!
```

34. What is self keyword?

Answer: self refers to the current instance of the class. It is used to access instance attributes and methods within the class. It must be the first parameter of instance methods (though it's passed automatically).

Example:

```
class Person:

    def __init__(self, name, age):

        self.name = name    # self.name is instance attribute

        self.age = age

    def introduce(self):

        # self refers to the object calling this method

        return f"I am {self.name}, age {self.age}"

    def birthday(self):

        self.age += 1    # modifying instance attribute

        return self    # returning self for method chaining
```

```
p = Person("Krishna", 25)
print(p.introduce()) # I am Krishna, age 25
```

self allows method chaining

```
class Builder:

    def __init__(self):

        self.items = []

    def add(self, item):

        self.items.append(item)

        return self # returning self

    def build(self):

        return self.items
```

```
result = Builder().add("a").add("b").add("c").build()
```

```
print(result) # ['a', 'b', 'c']
```

35. What are Class Variables and Instance Variables?

Answer:

- Class Variables: Shared by all instances, defined inside class but outside methods
- Instance Variables: Unique to each instance, defined inside `__init__` using `self`

Example:

```
class Employee:
    # Class variable - shared by all
    company = "TechCorp"
    employee_count = 0

    def __init__(self, name, salary):
        # Instance variables - unique to each object
        self.name = name
        self.salary = salary
        Employee.employee_count += 1 # modifying class variable

emp1 = Employee("Krishna", 50000)
emp2 = Employee("Ravi", 60000)

# Class variable - same for all
print(emp1.company)    # TechCorp
print(emp2.company)    # TechCorp
print(Employee.employee_count) # 2

# Instance variable - different for each
```



```
print(emp1.name) # Krishna
```

```
print(emp2.name) # Ravi
```

```
# Changing class variable affects all
```

```
Employee.company = "NewCorp"
```

```
print(emp1.company) # NewCorp
```

```
print(emp2.company) # NewCorp
```

```
# But assigning on instance creates instance variable
```

```
emp1.company = "OtherCorp"
```

```
print(emp1.company) # OtherCorp (instance variable)
```

```
print(emp2.company) # NewCorp (still class variable)
```

36. What are Static Methods?

Answer: Static methods belong to the class but don't access instance (self) or class (cls) data. They are utility functions that logically belong to the class. Defined using @staticmethod decorator.

Example:

```
class MathUtils:
```

```
    @staticmethod
```

```
    def add(a, b):
```

```
        return a + b
```

```
    @staticmethod
```

```
    def is_even(n):
```

```
        return n % 2 == 0
```

```
    @staticmethod
```

```

def factorial(n):
    if n <= 1:
        return 1
    return n * MathUtils.factorial(n - 1)

# Call without creating object
print(MathUtils.add(5, 3))    # 8
print(MathUtils.is_even(4))  # True
print(MathUtils.factorial(5)) # 120

# Can also call on instance (but unnecessary)
m = MathUtils()
print(m.add(2, 3)) # 5

class Validator:
    @staticmethod
    def is_valid_email(email):
        return "@" in email and "." in email

    @staticmethod
    def is_valid_age(age):
        return 0 < age < 150

print(Validator.is_valid_email("test@example.com")) # True
print(Validator.is_valid_age(25)) # True

```

37. What are Class Methods?

Answer: Class methods receive the class (cls) as the first argument instead of the instance (self). They can modify class state and are often used as alternative constructors. Defined using @classmethod decorator.

Example:

```
class Person:
```

```
    count = 0
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
        Person.count += 1
```

```
    @classmethod
```

```
    def from_string(cls, person_str):
```

```
        """Alternative constructor - creates Person from string"""
```

```
        name, age = person_str.split("-")
```

```
        return cls(name, int(age))
```

```
    @classmethod
```

```
    def from_dict(cls, data):
```

```
        """Alternative constructor - creates Person from dict"""
```

```
        return cls(data["name"], data["age"])
```

```
    @classmethod
```

```
    def get_count(cls):
```

```
        return cls.count
```

```
# Normal constructor
```

```
p1 = Person("Krishna", 25)
```

```
# Alternative constructors using class methods
p2 = Person.from_string("Ravi-30")
p3 = Person.from_dict({"name": "Priya", "age": 28})

print(p2.name, p2.age) # Ravi 30
print(p3.name, p3.age) # Priya 28
print(Person.get_count()) # 3
```

38. Difference between @staticmethod and @classmethod?

Answer:

Feature	@staticmethod	@classmethod	
-----	-----	-----	
First arg	None	cls (class itself)	
Access class	No	Yes	
Access inst	No	No	
Use case	Utility functions	Alternative constructors	
Modify class	No	Yes	

Example:

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius

    @classmethod
    def from_fahrenheit(cls, fahrenheit):
        # Has access to cls, can create new instance
        celsius = (fahrenheit - 32) * 5/9
        return cls(celsius)
```

```
@staticmethod
def is_boiling(celsius):
    # No access to cls or self, just a utility
    return celsius >= 100
```

```
# classmethod - creates instance
t = Temperature.from_fahrenheit(212)
print(t.celsius) # 100.0
```

```
# staticmethod - just a utility
print(Temperature.is_boiling(100)) # True
print(Temperature.is_boiling(50)) # False
```

39. What are Decorators?

Answer: Decorators are functions that modify the behavior of other functions or classes without changing their source code. They wrap a function, adding functionality before/after it. Defined using @ syntax.

Example:

```
# Basic decorator
def logger(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        result = func(*args, **kwargs)
        print(f"{func.__name__} returned {result}")
        return result
    return wrapper
```

```
@logger
```

```
def add(a, b):
```

```
    return a + b
```

```
add(3, 5)
```

```
# Calling add
```

```
# add returned 8
```

```
# Decorator with arguments
```

```
def repeat(times):
```

```
    def decorator(func):
```

```
        def wrapper(*args, **kwargs):
```

```
            for _ in range(times):
```

```
                result = func(*args, **kwargs)
```

```
            return result
```

```
        return wrapper
```

```
    return decorator
```

```
@repeat(3)
```

```
def greet(name):
```

```
    print(f"Hello {name}!")
```

```
greet("Krishna")
```

```
# Hello Krishna! (printed 3 times)
```

```
# Practical decorator - timing
```

```
import time
```

```
def timer(func):
```

```
    def wrapper(*args, **kwargs):
```

```
        start = time.time()
```

```

    result = func(*args, **kwargs)

    end = time.time()

    print(f"{func.__name__} took {end-start:.4f} seconds")

    return result

return wrapper

```

@timer

```

def slow_function():
    time.sleep(1)

```

```

slow_function() # slow_function took 1.00xx seconds

```

40. What are Python Modules?

Answer: A module is a Python file (.py) containing functions, classes, and variables. Modules help organize code into separate files for reusability. You import modules using the import statement.

Example:

--- mymath.py (creating a module) ---

```

def add(a, b):
    return a + b

```

```

def subtract(a, b):
    return a - b

```

PI = 3.14159

--- main.py (using the module) ---

Different ways to import

```
import mymath
print(mymath.add(5, 3)) # 8
print(mymath.PI)       # 3.14159
```

```
# Import specific items
from mymath import add, PI
print(add(5, 3))       # 8
```

```
# Import with alias
import mymath as mm
print(mm.subtract(10, 5)) # 5
```

```
# Import everything (not recommended)
from mymath import *
```

```
# Built-in modules
```

```
import os
import sys
import datetime
import random
```

```
print(os.getcwd())          # current directory
print(random.randint(1, 100)) # random number
print(datetime.datetime.now()) # current datetime
```

41. What are Python Packages?

Answer: A package is a directory containing multiple modules and a special `__init__.py` file. Packages help organize related modules into a directory hierarchy. They allow for structured, scalable codebases.

Example:

```
# Package structure:

# mypackage/
#   __init__.py
#   math_ops.py
#   string_ops.py
#   subpackage/
#       __init__.py
#       advanced.py


# --- mypackage/__init__.py ---
from .math_ops import add, multiply
from .string_ops import reverse


# --- mypackage/math_ops.py ---
def add(a, b):
    return a + b


def multiply(a, b):
    return a * b


# --- mypackage/string_ops.py ---
def reverse(s):
    return s[::-1]


# --- main.py (using the package) ---
from mypackage import add, reverse
print(add(5, 3))      # 8
print(reverse("hello")) # olleh
```

```
# Or import specific module
from mypackage.math_ops import multiply
print(multiply(4, 5)) # 20
```

42. Difference between Module and Package?

Answer:

Feature	Module	Package
What	Single .py file	Directory with modules
Contains	Functions, classes, vars	Multiple modules
File	filename.py	folder/___init___.py
Import	import module	from package import module
Structure	Flat (single file)	Hierarchical (directories)

Example:

```
# Module: a single file
# utils.py
def greet(name):
    return f"Hello {name}"
```

```
# Package: a directory with __init__.py
# my_project/
#   __init__.py
#   utils.py
#   models.py
#   services/
#     __init__.py
#     auth.py
```

```
import utils          # importing module  
from my_project import utils # importing module from package
```

43. What is Virtual Environment?

Answer: A virtual environment is an isolated Python environment that allows you to install packages specific to a project without affecting the global Python installation. Each project can have its own dependencies and versions.

Example:

```
# Creating virtual environment
```

```
# python -m venv myenv
```

```
# Activating (Windows)
```

```
# myenv\Scripts\activate
```

```
# Activating (Linux/Mac)
```

```
# source myenv/bin/activate
```

```
# Installing packages in venv
```

```
# pip install flask
```

```
# pip install requests==2.28.0
```

```
# Freeze dependencies
```

```
# pip freeze > requirements.txt
```

```
# Install from requirements
```

```
# pip install -r requirements.txt
```

```
# Deactivate
```

```
# deactivate
```

```
# requirements.txt example:
```

```
# flask==2.3.0
```

```
# requests==2.28.0
```

```
# sqlalchemy==2.0.0
```

44. Why do we use pip?

Answer: pip (Python Install Packages) is the package manager for Python. It downloads and installs packages from PyPI (Python Package Index). It manages dependencies, versions, and package updates.

Example:

```
# Install a package
```

```
# pip install requests
```

```
# Install specific version
```

```
# pip install flask==2.3.0
```

```
# Upgrade package
```

```
# pip install --upgrade pip
```

```
# Uninstall package
```

```
# pip uninstall requests
```

```
# List installed packages
```

```
# pip list
```

```
# Show package info
```

```
# pip show flask
```

```
# Save dependencies
```

```
# pip freeze > requirements.txt
```

```
# Install from file
```

```
# pip install -r requirements.txt
```

```
# Search for package (deprecated, use PyPI website)
```

```
# pip search flask
```

```
# Using pip in code
```

```
import subprocess
```

```
import sys
```

```
# subprocess.check_call([sys.executable, "-m", "pip", "install", "requests"])
```

45. What is PEP 8?

Answer: PEP 8 is the official Python style guide. It defines coding conventions for writing readable, consistent Python code. Key rules include indentation, naming conventions, line length, imports, and whitespace usage.

Example:

```
# PEP 8 Rules with examples:
```

```
# 1. Indentation: 4 spaces
```

```
def my_function():
```

```
    if True:
```

```
        print("correct")
```

2. Line length: max 79 characters

Break long lines

```
result = (first_variable
          + second_variable
          - third_variable)
```

3. Naming conventions

```
my_variable = 10      # snake_case for variables
```

```
def my_function():    # snake_case for functions
    pass
```

```
class MyClass:        # PascalCase for classes
    pass
```

```
MAX_VALUE = 100       # UPPER_CASE for constants
```

4. Imports at top, one per line

```
import os
```

```
import sys
```

```
from collections import defaultdict
```

5. Spaces around operators

```
x = 5 + 3            # Good
```

```
# x=5+3              # Bad
```

6. Two blank lines before top-level functions/classes

One blank line between methods

46. What is Python GIL?

Answer: GIL (Global Interpreter Lock) is a mutex in CPython that allows only one thread to execute Python bytecode at a time. This means even on multi-core systems, only one thread runs Python code at any given moment. It simplifies memory management but limits true parallelism for CPU-bound tasks.

Example:

```
import threading
import time

# GIL limits CPU-bound multithreading
def cpu_bound_task(n):
    count = 0
    for i in range(n):
        count += i
    return count

# Single thread
start = time.time()
cpu_bound_task(10000000)
cpu_bound_task(10000000)
print(f"Single thread: {time.time()-start:.2f}s")

# Two threads (not faster due to GIL)
start = time.time()
t1 = threading.Thread(target=cpu_bound_task, args=(10000000,))
t2 = threading.Thread(target=cpu_bound_task, args=(10000000,))
t1.start()
t2.start()
t1.join()
t2.join()
print(f"Two threads: {time.time()-start:.2f}s")
```

```
# Solution: Use multiprocessing for CPU-bound tasks
# GIL doesn't affect I/O-bound tasks (file, network operations)
```

47. What is Multithreading?

Answer: Multithreading allows multiple threads to run concurrently within a single process. Threads share the same memory space. It's useful for I/O-bound tasks (file operations, network calls, database queries) but limited for CPU-bound tasks due to GIL.

Example:

```
import threading
import time

def download_file(filename):
    print(f"Starting download: {filename}")
    time.sleep(2) # simulating I/O
    print(f"Finished download: {filename}")

# Without threading (sequential) - takes 6 seconds
# download_file("file1.pdf")
# download_file("file2.pdf")
# download_file("file3.pdf")

# With threading (concurrent) - takes ~2 seconds
threads = []
files = ["file1.pdf", "file2.pdf", "file3.pdf"]

for f in files:
```



```

t = threading.Thread(target=download_file, args=(f,))
threads.append(t)
t.start()

for t in threads:
    t.join() # wait for all to finish

print("All downloads complete!")

# Thread with return value using shared data
results = {}

def fetch_data(url, results, index):
    time.sleep(1)
    results[index] = f"Data from {url}"

t1 = threading.Thread(target=fetch_data, args=("api/users", results, 0))
t2 = threading.Thread(target=fetch_data, args=("api/posts", results, 1))
t1.start()
t2.start()
t1.join()
t2.join()
print(results)

```

48. What is Multiprocessing?

Answer: Multiprocessing creates separate processes, each with its own Python interpreter and memory space. It bypasses the GIL, enabling true parallelism. Best for CPU-bound tasks (calculations, data processing).

Example:

```
import multiprocessing
import time

def cpu_heavy_task(n):
    """CPU-bound task"""
    return sum(i * i for i in range(n))

# Sequential
start = time.time()
results = [cpu_heavy_task(10000000) for _ in range(4)]
print(f"Sequential: {time.time()-start:.2f}s")

# Multiprocessing
start = time.time()
with multiprocessing.Pool(processes=4) as pool:
    results = pool.map(cpu_heavy_task, [10000000]*4)
print(f"Multiprocessing: {time.time()-start:.2f}s")

# Using Process class
def worker(name):
    print(f"Worker {name} started (PID: {multiprocessing.current_process().pid})")
    time.sleep(2)
    print(f"Worker {name} finished")

if __name__ == "__main__":
    processes = []
    for i in range(3):
        p = multiprocessing.Process(target=worker, args=(i,))
        processes.append(p)
```

```
p.start()
```

```
for p in processes:
```

```
    p.join()
```

49. Difference between Thread and Process?

Answer:

Feature	Thread	Process	
-----	-----	-----	
Memory	Shared memory	Separate memory	
Overhead	Lightweight	Heavyweight	
Communication	Easy (shared variables)	IPC needed (pipes, queues)	
GIL	Affected by GIL	Not affected by GIL	
Best for	I/O-bound tasks	CPU-bound tasks	
Creation	Fast	Slower	
Crash impact	Can crash whole process	Other processes unaffected	

Example:

```
import threading
```

```
import multiprocessing
```

```
import time
```

```
def task(name):
```

```
    time.sleep(1)
```

```
    print(f"Task {name} done")
```

```
# Threading - for I/O bound (network, file operations)
```

```
print("--- Threading ---")
```

```

threads = [threading.Thread(target=task, args=(i,)) for i in range(3)]
for t in threads:
    t.start()
for t in threads:
    t.join()

# Multiprocessing - for CPU bound (heavy calculations)
print("--- Multiprocessing ---")
if __name__ == "__main__":
    processes = [multiprocessing.Process(target=task, args=(i,)) for i in range(3)]
    for p in processes:
        p.start()
    for p in processes:
        p.join()

```

50. What is Async Programming?

Answer: Asynchronous programming allows tasks to run concurrently without blocking. Instead of waiting for one task to finish, the program can start another task. Python uses async/await syntax with the asyncio module. It's ideal for I/O-bound operations.

Example:

```

import asyncio

# Async function (coroutine)
async def fetch_data(url, delay):
    print(f"Fetching {url}...")
    await asyncio.sleep(delay) # non-blocking wait
    print(f"Got data from {url}")

```

```

    return f"Data from {url}"

# Running multiple async tasks concurrently
async def main():
    # These run concurrently, not sequentially
    tasks = [
        fetch_data("api/users", 2),
        fetch_data("api/posts", 1),
        fetch_data("api/comments", 3),
    ]
    results = await asyncio.gather(*tasks)
    print(results)

# Run the async program
asyncio.run(main())

# Total time: ~3 seconds (not 6 seconds)

# Comparison:
# Synchronous: task1(2s) + task2(1s) + task3(3s) = 6s total
# Asynchronous: all run together = ~3s total (longest task)

```

51. What is Asyncio?

Answer: asyncio is Python's built-in library for writing asynchronous code using async/await syntax. It provides an event loop that manages and runs coroutines, handles I/O operations, and enables concurrent execution without threads.

Example:

```

import asyncio

# Basic coroutine

```

```
async def say_hello(name, delay):
```

```
    await asyncio.sleep(delay)
```

```
    print(f"Hello, {name}!")
```

```
# Event loop runs coroutines
```

```
async def main():
```

```
    # Run sequentially
```

```
    await say_hello("Krishna", 1)
```

```
    await say_hello("Ravi", 1)
```

```
# Run concurrently using gather
```

```
await asyncio.gather(
```

```
    say_hello("A", 2),
```

```
    say_hello("B", 1),
```

```
    say_hello("C", 3),
```

```
)
```

```
asyncio.run(main())
```

```
# asyncio.create_task for concurrent execution
```

```
async def main_tasks():
```

```
    task1 = asyncio.create_task(say_hello("Task1", 2))
```

```
    task2 = asyncio.create_task(say_hello("Task2", 1))
```

```
    await task1
```

```
    await task2
```

```
asyncio.run(main_tasks())
```

52. Difference between synchronous and asynchronous programming?

Answer:

Feature	Synchronous	Asynchronous	
-----	-----	-----	
Execution	One task at a time	Multiple tasks concurrently	
Blocking	Blocks until complete	Non-blocking	
Performance	Slower for I/O tasks	Faster for I/O tasks	
Complexity	Simple	More complex	
Use case	CPU-bound, simple scripts I/O-bound, web servers		

Example:

```
import asyncio
import time
```

```
# Synchronous - blocks on each call
```

```
def sync_tasks():
    start = time.time()
    time.sleep(2) # task 1
    time.sleep(2) # task 2
    time.sleep(2) # task 3
    print(f"Sync total: {time.time()-start:.1f}s") # ~6s
```

```
sync_tasks()
```

```
# Asynchronous - runs concurrently
```

```
async def async_tasks():
    start = time.time()
    await asyncio.gather(
        asyncio.sleep(2), # task 1
        asyncio.sleep(2), # task 2
```

```
        asyncio.sleep(2), # task 3
    )
    print(f'Async total: {time.time()-start:.1f}s") # ~2s

asyncio.run(async_tasks())
```

53. What is File Handling?

Answer: File handling allows reading from and writing to files. Python uses the built-in `open()` function to work with files. The `with` statement is recommended as it automatically closes the file.

Example:

```
# Writing to a file
with open("example.txt", "w") as f:
    f.write("Hello World\n")
    f.write("Python File Handling\n")
```

```
# Reading entire file
with open("example.txt", "r") as f:
    content = f.read()
    print(content)
```

```
# Reading line by line
with open("example.txt", "r") as f:
    for line in f:
        print(line.strip())
```

```
# Reading lines into list
with open("example.txt", "r") as f:
```



```
lines = f.readlines()

print(lines) # ['Hello World\n', 'Python File Handling\n']
```

```
# Appending to file

with open("example.txt", "a") as f:
    f.write("New line appended\n")
```

```
# Reading and writing (r+)

with open("example.txt", "r+") as f:
    content = f.read()
    f.write("Added at end")
```

54. Different file modes in Python?

Answer:

Mode	Description
'r'	Read only (default). Error if file not exists
'w'	Write only. Creates/overwrites file
'a'	Append. Creates file if not exists
'x'	Exclusive create. Error if file exists
'r+'	Read and write. File must exist
'w+'	Write and read. Creates/overwrites
'a+'	Append and read. Creates if not exists
'b'	Binary mode (add to above: 'rb', 'wb')
't'	Text mode (default)

Example:

```
# 'w' - write (creates new / overwrites)
```

```
with open("test.txt", "w") as f:
```

```
    f.write("Fresh content")
```

```
# 'a' - append (adds to end)
```

```
with open("test.txt", "a") as f:
```

```
    f.write("\nAppended content")
```

```
# 'x' - exclusive create (fails if exists)
```

```
try:
```

```
    with open("new_file.txt", "x") as f:
```

```
        f.write("Brand new file")
```

```
except FileExistsError:
```

```
    print("File already exists!")
```

```
# 'rb' - read binary (images, PDFs)
```

```
with open("image.png", "rb") as f:
```

```
    binary_data = f.read()
```

```
# 'wb' - write binary
```

```
with open("copy.png", "wb") as f:
```

```
    f.write(binary_data)
```

55. What is JSON?

Answer: JSON (JavaScript Object Notation) is a lightweight data interchange format. It's human-readable, language-independent, and commonly used for APIs and configuration files. Python's json module handles JSON serialization/deserialization.

Example:

```
import json
```

```
# Python dict to JSON string (serialization)
```

```
data = {  
    "name": "Krishna",  
    "age": 25,  
    "skills": ["Python", "FastAPI", "AWS"],  
    "active": True  
}
```

```
json_string = json.dumps(data, indent=2)  
print(json_string)
```

```
# JSON string to Python dict (deserialization)
```

```
json_str = '{"name": "Krishna", "age": 25}'  
python_dict = json.loads(json_str)  
print(python_dict["name"]) # Krishna
```

```
# Write JSON to file
```

```
with open("data.json", "w") as f:  
    json.dump(data, f, indent=2)
```

```
# Read JSON from file
```

```
with open("data.json", "r") as f:  
    loaded_data = json.load(f)  
    print(loaded_data)
```

56. How do you read and write JSON files?

Answer: Use `json.dump()` to write and `json.load()` to read JSON files. For strings, use `json.dumps()` and `json.loads()`.

Example:

```
import json

# Writing JSON file
employees = [
    {"name": "Krishna", "role": "Developer", "salary": 50000},
    {"name": "Ravi", "role": "Designer", "salary": 45000},
]

with open("employees.json", "w") as f:
    json.dump(employees, f, indent=4)

# Reading JSON file
with open("employees.json", "r") as f:
    data = json.load(f)

for emp in data:
    print(f"{emp['name']} - {emp['role']}")

# Handling complex types
from datetime import datetime

class DateEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, datetime):
            return obj.isoformat()
        return super().default(obj)

event = {"name": "Meeting", "date": datetime.now()}
```

```
json_str = json.dumps(event, cls=DateEncoder)
print(json_str)
```

57. What is Pickling and Unpickling?

Answer:

- Pickling: Serializing Python objects into byte streams (binary format)
- Unpickling: Deserializing byte streams back into Python objects

Unlike JSON, pickle can handle complex Python objects (classes, functions).

Example:

```
import pickle

# Pickling (serializing)
data = {
    "name": "Krishna",
    "scores": [95, 87, 92],
    "info": {"city": "Hyderabad", "active": True}
}

# Save to file
with open("data.pkl", "wb") as f:
    pickle.dump(data, f)

# Unpickling (deserializing)
with open("data.pkl", "rb") as f:
    loaded = pickle.load(f)
print(loaded) # same as original data
```

```
# Pickle complex objects

class Person:

    def __init__(self, name, age):

        self.name = name

        self.age = age
```

```
person = Person("Krishna", 25)
```

```
# Serialize to bytes

pickled = pickle.dumps(person)

# Deserialize from bytes

unpickled = pickle.loads(pickled)

print(unpickled.name) # Krishna
```

WARNING: Never unpickle data from untrusted sources (security risk!)

58. What is Deep Copy and Shallow Copy?

Answer:

- Shallow Copy: Creates new object but references the same nested objects. Changes to nested objects affect both.
- Deep Copy: Creates completely independent copy. All nested objects are also copied.

Example:

```
import copy

# Original

original = [[1, 2, 3], [4, 5, 6]]
```

```
# Shallow copy
shallow = copy.copy(original)
shallow[0][0] = 999
print(original[0][0]) # 999! (nested list is shared)
```

```
# Deep copy
original2 = [[1, 2, 3], [4, 5, 6]]
deep = copy.deepcopy(original2)
deep[0][0] = 999
print(original2[0][0]) # 1 (completely independent)
```

```
# Practical example
class Config:
    def __init__(self):
        self.settings = {"debug": True, "db": {"host": "localhost"}}
```

```
config1 = Config()
config2 = copy.copy(config1)    # shallow
config3 = copy.deepcopy(config1) # deep
```

```
config2.settings["db"]["host"] = "remote"
print(config1.settings["db"]["host"]) # "remote" (shared!)
```

```
config3.settings["db"]["host"] = "other"
print(config1.settings["db"]["host"]) # "remote" (unaffected by deep copy)
```

```
# List copy methods
lst = [1, [2, 3]]
a = lst[:]    # shallow copy
b = list(lst) # shallow copy
```

```
c = copy.deepcopy(lst) # deep copy
```

59. Difference between is and ==?

Answer:

- == (equality): Checks if values are equal
- is (identity): Checks if both variables point to the same object in memory

Example:

```
# == checks value equality
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b) # True (same values)
print(a is b) # False (different objects in memory)
```

```
# is checks identity (same object)
c = a
print(a is c) # True (same object)
print(id(a) == id(c)) # True
```

```
# Integer caching (-5 to 256)
x = 256
y = 256
print(x is y) # True (cached)
```

```
x = 257
y = 257
print(x is y) # False (not cached, may vary)
```



```
# None comparison - always use 'is'
value = None
print(value is None) # True (correct way)
print(value == None) # True (works but not recommended)
```

```
# String interning
s1 = "hello"
s2 = "hello"
print(s1 is s2) # True (Python interns short strings)
```

60. What are Python Built-in Functions?

Answer: Built-in functions are pre-defined functions available without importing any module. Python has 70+ built-in functions like print(), len(), type(), range(), input(), etc.

Example:

```
# Common built-in functions
print(len("hello"))      # 5
print(type(42))          # <class 'int'>
print(abs(-10))          # 10
print(max(1, 5, 3))      # 5
print(min(1, 5, 3))      # 1
print(sum([1, 2, 3, 4])) # 10
print(round(3.14159, 2)) # 3.14
print(sorted([3, 1, 2])) # [1, 2, 3]
print(reversed([1, 2, 3])) # iterator

# Type conversion
print(int("42"))         # 42
```

```
print(float("3.14"))    # 3.14
print(str(100))          # "100"
print(list("abc"))       # ['a', 'b', 'c']
print(bool(0))           # False
print(bool(1))           # True
```

Useful functions

```
print(isinstance(5, int)) # True
print(id(42))             # memory address
print(dir([]))            # list methods
print(help(len))          # documentation
```

range, enumerate, zip

```
print(list(range(5)))     # [0, 1, 2, 3, 4]
print(list(enumerate("abc")))# [(0,'a'), (1,'b'), (2,'c')]
print(list(zip([1,2], [3,4])))# [(1,3), (2,4)]
```

61. What is map()?

Answer: map() applies a function to every item in an iterable and returns a map object (iterator). It's used for transforming data without explicit loops.

Syntax: map(function, iterable)

Example:

```
# Basic map
nums = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x**2, nums))
print(squared) # [1, 4, 9, 16, 25]
```

With named function

```

def to_upper(s):
    return s.upper()

words = ["hello", "world", "python"]
result = list(map(to_upper, words))
print(result) # ['HELLO', 'WORLD', 'PYTHON']

# Multiple iterables
a = [1, 2, 3]
b = [10, 20, 30]
sums = list(map(lambda x, y: x + y, a, b))
print(sums) # [11, 22, 33]

# Convert string numbers to int
str_nums = ["1", "2", "3", "4"]
int_nums = list(map(int, str_nums))
print(int_nums) # [1, 2, 3, 4]

```

62. What is filter()?

Answer: filter() filters elements from an iterable based on a function that returns True/False. Only elements where the function returns True are kept.

Syntax: filter(function, iterable)

Example:

```

# Filter even numbers
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens) # [2, 4, 6, 8, 10]

```

```
# Filter non-empty strings
words = ["hello", "", "world", "", "python"]
non_empty = list(filter(None, words))
print(non_empty) # ['hello', 'world', 'python']
```

```
# Filter with named function
```

```
def is_adult(age):
    return age >= 18
```

```
ages = [12, 18, 25, 8, 30, 15]
adults = list(filter(is_adult, ages))
print(adults) # [18, 25, 30]
```

```
# Combine map and filter
```

```
nums = range(1, 21)
```

```
# Get squares of even numbers
```

```
result = list(map(lambda x: x**2, filter(lambda x: x % 2 == 0, nums)))
print(result) # [4, 16, 36, 64, 100, 144, 196, 256, 324, 400]
```

63. What is reduce()?

Answer: reduce() applies a function cumulatively to items in an iterable, reducing it to a single value. It's in the functools module. It takes the first two elements, applies the function, then takes the result with the next element, and so on.

Syntax: `functools.reduce(function, iterable, initial)`

Example:

```
from functools import reduce
```

```
# Sum of all numbers
```

```
nums = [1, 2, 3, 4, 5]
```

```
total = reduce(lambda x, y: x + y, nums)
```

```
print(total) # 15 (((1+2)+3)+4)+5)
```

```
# Find maximum
```

```
maximum = reduce(lambda x, y: x if x > y else y, nums)
```

```
print(maximum) # 5
```

```
# Product of all numbers
```

```
product = reduce(lambda x, y: x * y, nums)
```

```
print(product) # 120
```

```
# With initial value
```

```
total = reduce(lambda x, y: x + y, nums, 100)
```

```
print(total) # 115 (100 + 1+2+3+4+5)
```

```
# Flatten nested list
```

```
nested = [[1, 2], [3, 4], [5, 6]]
```

```
flat = reduce(lambda x, y: x + y, nested)
```

```
print(flat) # [1, 2, 3, 4, 5, 6]
```

64. What is zip()?

Answer: zip() combines multiple iterables element-wise into tuples. It stops when the shortest iterable is exhausted. Returns a zip object (iterator).

Example:

```
# Basic zip
```

```
names = ["Krishna", "Ravi", "Priya"]
ages = [25, 30, 28]
cities = ["Hyderabad", "Mumbai", "Delhi"]

combined = list(zip(names, ages, cities))
print(combined)
# [('Krishna', 25, 'Hyderabad'), ('Ravi', 30, 'Mumbai'), ('Priya', 28, 'Delhi')]

# Create dictionary from two lists
keys = ["name", "age", "city"]
values = ["Krishna", 25, "Hyderabad"]
person = dict(zip(keys, values))
print(person) # {'name': 'Krishna', 'age': 25, 'city': 'Hyderabad'}

# Unzip using *
pairs = [(1, 'a'), (2, 'b'), (3, 'c')]
numbers, letters = zip(*pairs)
print(numbers) # (1, 2, 3)
print(letters) # ('a', 'b', 'c')

# Parallel iteration
for name, age in zip(names, ages):
    print(f"{name} is {age} years old")

# zip_longest (includes all elements)
from itertools import zip_longest
a = [1, 2, 3]
b = [4, 5]
print(list(zip_longest(a, b, fillvalue=0))) # [(1,4), (2,5), (3,0)]
```

65. What is enumerate()?

Answer: enumerate() adds a counter to an iterable and returns enumerate object with (index, value) pairs. It's useful when you need both index and value while iterating.

Example:

```
# Basic enumerate
```

```
fruits = ["apple", "banana", "cherry"]
```

```
for index, fruit in enumerate(fruits):
```

```
    print(f"{index}: {fruit}")
```

```
# 0: apple
```

```
# 1: banana
```

```
# 2: cherry
```

```
# Custom start index
```

```
for index, fruit in enumerate(fruits, start=1):
```

```
    print(f"{index}: {fruit}")
```

```
# 1: apple
```

```
# 2: banana
```

```
# 3: cherry
```

```
# Convert to list/dict
```

```
print(list(enumerate(fruits)))    # [(0,'apple'), (1,'banana'), (2,'cherry')]
```

```
print(dict(enumerate(fruits)))    # {0:'apple', 1:'banana', 2:'cherry'}
```

```
# Practical: find index of element
```

```
scores = [85, 92, 78, 95, 88]
```

```
max_score = max(scores)
```

```
max_index = next(i for i, s in enumerate(scores) if s == max_score)
```

```
print(f"Highest score {max_score} at index {max_index}") # 95 at index 3
```

66. What is any() and all()?

Answer:

- any(): Returns True if ANY element in iterable is truthy
- all(): Returns True if ALL elements in iterable are truthy

Example:

```
# any() - at least one True
print(any([False, False, True])) # True
print(any([False, False, False])) # False
print(any([0, 0, 1]))           # True
print(any([]))                  # False
```

```
# all() - all must be True
print(all([True, True, True])) # True
print(all([True, False, True])) # False
print(all([1, 2, 3]))          # True
print(all([]))                 # True
```

```
# Practical examples
```

```
ages = [18, 25, 30, 16, 22]
```

```
# Check if any minor
```

```
has_minor = any(age < 18 for age in ages)
```

```
print(f"Has minor: {has_minor}") # True
```

```
# Check if all adults
```



```
all_adults = all(age >= 18 for age in ages)
print(f"All adults: {all_adults}") # False
```

```
# Validate data
```

```
passwords = ["abc123", "xyz789", ""]
all_valid = all(len(p) >= 6 for p in passwords)
print(f"All valid: {all_valid}") # False
```

```
# Check if any file exists
```

```
import os
files = ["test.txt", "data.json", "config.yml"]
any_exists = any(os.path.exists(f) for f in files)
```

67. What are Context Managers?

Answer: Context managers handle setup and cleanup of resources automatically. They ensure resources (files, connections, locks) are properly released even if errors occur. Implemented using `__enter__` and `__exit__` methods or the `contextlib` module.

Example:

```
# Using context manager (with statement)
with open("file.txt", "w") as f:
    f.write("Hello")
# File is automatically closed after this block
```

```
# Custom context manager using class
```

```
class DatabaseConnection:
```

```
    def __init__(self, host):
        self.host = host
```

```

def __enter__(self):
    print(f"Connecting to {self.host}")
    self.connection = f"Connection to {self.host}"
    return self.connection

def __exit__(self, exc_type, exc_val, exc_tb):
    print("Closing connection")
    self.connection = None
    return False # don't suppress exceptions

with DatabaseConnection("localhost") as conn:
    print(f"Using: {conn}")
# Connecting to localhost
# Using: Connection to localhost
# Closing connection

# Using contextlib decorator
from contextlib import contextmanager

@contextmanager
def timer():
    import time
    start = time.time()
    yield
    end = time.time()
    print(f"Elapsed: {end-start:.2f}s")

with timer():
    import time
    time.sleep(1)

```

Elapsed: 1.00s

68. What is the with statement?

Answer: The with statement simplifies resource management by automatically handling setup and cleanup. It works with context managers and ensures resources are properly released even if exceptions occur.

Example:

```
# File handling with 'with'
```

```
with open("output.txt", "w") as f:
```

```
    f.write("Line 1\n")
```

```
    f.write("Line 2\n")
```

```
# f is automatically closed here
```

```
# Without 'with' (manual handling)
```

```
f = open("output.txt", "r")
```

```
try:
```

```
    content = f.read()
```

```
finally:
```

```
    f.close() # must manually close
```

```
# Multiple context managers
```

```
with open("input.txt", "r") as infile, open("output.txt", "w") as outfile:
```

```
    for line in infile:
```

```
        outfile.write(line.upper())
```

```
# with threading lock
```

```
import threading
```

```
lock = threading.Lock()
```

with lock:

```
# critical section - lock automatically released  
print("Thread-safe operation")
```

Nested with

```
with open("file1.txt", "r") as f1:  
    with open("file2.txt", "w") as f2:  
        f2.write(f1.read())
```

69. What are Regular Expressions?

Answer: Regular expressions (regex) are patterns used to match, search, and manipulate strings. Python's re module provides regex support. Common uses: validation, searching, replacing text patterns.

Example:

```
import re  
  
# Basic matching  
text = "My phone is 123-456-7890"  
pattern = r"\d{3}-\d{3}-\d{4}"  
match = re.search(pattern, text)  
print(match.group()) # 123-456-7890  
  
# Common patterns  
# \d - digit, \w - word char, \s - whitespace  
# + one or more, * zero or more, ? zero or one  
# ^ start, $ end, . any char  
  
# Find all emails
```

```
text = "Contact us at test@example.com or admin@site.org"
emails = re.findall(r"[\w.]+\@[ \w.]+", text)
print(emails) # ['test@example.com', 'admin@site.org']
```

Replace

```
text = "Hello World"
result = re.sub(r"World", "Python", text)
print(result) # Hello Python
```

Validate email

```
def is_valid_email(email):
    pattern = r"^\[w.\]+\@[ \w.\]+\.[ \w.\]+$"
    return bool(re.match(pattern, email))

print(is_valid_email("test@example.com")) # True
print(is_valid_email("invalid"))          # False
```

Groups

```
date = "2024-01-15"
match = re.match(r"(\d{4})-(\d{2})-(\d{2})", date)
print(match.group(1)) # 2024
print(match.group(2)) # 01
print(match.group(3)) # 15
```

Split

```
text = "one,two;three four"
parts = re.split(r"[,; \s]+", text)
print(parts) # ['one', 'two', 'three', 'four']
```

70. What is Unit Testing in Python?

Answer: Unit testing is testing individual units/functions of code in isolation. Python has built-in unittest module and popular third-party pytest framework. Tests verify that code works correctly.

Example:

```
# --- calculator.py ---
```

```
def add(a, b):
```

```
    return a + b
```

```
def divide(a, b):
```

```
    if b == 0:
```

```
        raise ValueError("Cannot divide by zero")
```

```
    return a / b
```

```
# --- test_calculator.py (using unittest) ---
```

```
import unittest
```

```
class TestCalculator(unittest.TestCase):
```

```
    def test_add(self):
```

```
        self.assertEqual(add(2, 3), 5)
```

```
        self.assertEqual(add(-1, 1), 0)
```

```
    def test_divide(self):
```

```
        self.assertEqual(divide(10, 2), 5)
```

```
        self.assertAlmostEqual(divide(1, 3), 0.333, places=2)
```

```
    def test_divide_by_zero(self):
```

```
        with self.assertRaises(ValueError):
```

```
            divide(10, 0)
```

```

if __name__ == "__main__":
    unittest.main()

# --- test_calculator_pytest.py (using pytest) ---
# Simpler syntax with pytest
def test_add():
    assert add(2, 3) == 5

def test_divide():
    assert divide(10, 2) == 5

def test_divide_by_zero():
    import pytest
    with pytest.raises(ValueError):
        divide(10, 0)

# Run: pytest test_calculator_pytest.py

```

71. What is PyTest?

Answer: PyTest is a popular third-party testing framework for Python. It's simpler than unittest, uses plain assert statements, supports fixtures, parameterized tests, and has a rich plugin ecosystem.

Example:

```

# Install: pip install pytest

# --- test_app.py ---
import pytest

```

```
# Simple test
def test_addition():
    assert 1 + 1 == 2

# Test with fixture
@pytest.fixture
def sample_list():
    return [1, 2, 3, 4, 5]

def test_list_length(sample_list):
    assert len(sample_list) == 5

def test_list_sum(sample_list):
    assert sum(sample_list) == 15

# Parameterized test
@pytest.mark.parametrize("input,expected", [
    (2, 4),
    (3, 9),
    (4, 16),
])
def test_square(input, expected):
    assert input ** 2 == expected

# Test exceptions
def test_zero_division():
    with pytest.raises(ZeroDivisionError):
        1 / 0

# Run: pytest test_app.py -v
```


72. What is Logging?

Answer: Logging is a way to track events in your application. Unlike print(), logging provides severity levels, timestamps, output to files, and is configurable. Levels: DEBUG, INFO, WARNING, ERROR, CRITICAL.

Example:

```
import logging

# Basic configuration
logging.basicConfig(
    level=logging.DEBUG,
    format="%(asctime)s - %(levelname)s - %(message)s",
    filename="app.log"
)

logger = logging.getLogger(__name__)

# Logging levels
logger.debug("Debug message - detailed info")
logger.info("Info - general information")
logger.warning("Warning - something unexpected")
logger.error("Error - something failed")
logger.critical("Critical - program may crash")

# Practical usage
def divide(a, b):
    logger.info(f"Dividing {a} by {b}")
    try:
```

```
    result = a / b
    logger.debug(f"Result: {result}")
    return result
except ZeroDivisionError:
    logger.error("Division by zero attempted!")
    return None
```

```
divide(10, 2) # logs info and debug
divide(10, 0) # logs info and error
```

```
# Log to both file and console
handler = logging.StreamHandler()
handler.setLevel(logging.WARNING)
logger.addHandler(handler)
```

73. What is Monkey Patching?

Answer: Monkey patching is dynamically modifying a class or module at runtime. It's used to change behavior of existing code without modifying the source. Common in testing to mock dependencies.

Example:

```
# Basic monkey patching
class MyClass:
    def greet(self):
        return "Hello"

obj = MyClass()
print(obj.greet()) # Hello
```

```
# Monkey patch the method
```

```
def new_greet(self):  
    return "Hi there!"
```

```
MyClass.greet = new_greet
```

```
print(obj.greet()) # Hi there!
```

```
# Practical: Monkey patching in tests
```

```
import requests
```

```
def get_user_data(user_id):  
    response = requests.get(f"https://api.example.com/users/{user_id}")  
    return response.json()
```

```
# In tests - mock the requests.get
```

```
class MockResponse:
```

```
    def json(self):  
        return {"id": 1, "name": "Test User"}
```

```
def mock_get(url):  
    return MockResponse()
```

```
# Patch
```

```
original_get = requests.get
```

```
requests.get = mock_get
```

```
print(get_user_data(1)) # {'id': 1, 'name': 'Test User'}
```

```
# Restore
```

```
requests.get = original_get
```

```
# Better way: using unittest.mock
```

```
from unittest.mock import patch
```

```
@patch('requests.get')
```

```
def test_get_user(mock_get):
```

```
    mock_get.return_value.json.return_value = {"id": 1, "name": "Mock"}
```

```
    result = get_user_data(1)
```

```
    assert result["name"] == "Mock"
```

74. What is Duck Typing?

Answer: "If it walks like a duck and quacks like a duck, it's a duck." In Python, we don't check the type of an object; we check if it has the required method/attribute. If an object has the right methods, it can be used regardless of its actual type.

Example:

```
# Duck typing - no type checking needed
```

```
class Duck:
```

```
    def quack(self):
```

```
        print("Quack!")
```

```
    def walk(self):
```

```
        print("Walking like a duck")
```

```
class Person:
```

```
    def quack(self):
```

```
        print("I'm quacking like a duck!")
```

```
    def walk(self):
```

```
        print("I'm walking like a duck!")
```

```
# This function works with ANY object that has quack() and walk()
```

```
def duck_test(animal):
```

```
    animal.quack()
```

```
    animal.walk()
```

```
duck_test(Duck())    # Works!
```

```
duck_test(Person()) # Also works! (duck typing)
```

```
# Practical example
```

```
class FileWriter:
```

```
    def write(self, data):
```

```
        with open("out.txt", "w") as f:
```

```
            f.write(data)
```

```
class ConsoleWriter:
```

```
    def write(self, data):
```

```
        print(data)
```

```
def save_report(writer, data):
```

```
    writer.write(data) # works with any object that has write()
```

```
save_report(FileWriter(), "Report data")
```

```
save_report(ConsoleWriter(), "Report data")
```

```
# Built-in duck typing: len() works on anything with __len__
```

```
class MyCollection:
```

```
    def __len__(self):
```

```
        return 42
```

```
print(len(MyCollection())) # 42
```

75. What are Dataclasses?

Answer: Dataclasses (Python 3.7+) are a decorator that automatically generates `__init__`, `__repr__`, `__eq__`, and other methods for classes that primarily store data. Reduces boilerplate code.

Example:

```
from dataclasses import dataclass, field

@dataclass
class Employee:
    name: str
    age: int
    salary: float
    department: str = "Engineering" # default value
```

```
# Auto-generated __init__, __repr__, __eq__
emp = Employee("Krishna", 25, 50000)

print(emp) # Employee(name='Krishna', age=25, salary=50000,
department='Engineering')
```

```
# Comparison
emp2 = Employee("Krishna", 25, 50000)
print(emp == emp2) # True (auto __eq__)
```

```
# With field and default_factory
```

```
@dataclass
class Team:
```

```

name: str
members: list = field(default_factory=list)
size: int = field(init=False)

def __post_init__(self):
    self.size = len(self.members)

team = Team("Backend", ["Krishna", "Ravi"])
print(team)    # Team(name='Backend', members=['Krishna', 'Ravi'], size=2)

# Frozen (immutable) dataclass
@dataclass(frozen=True)
class Point:
    x: float
    y: float

p = Point(1.0, 2.0)
# p.x = 3.0 # FrozenInstanceError!

```

76. What is Type Hinting?

Answer: Type hints (Python 3.5+) allow you to annotate variable types, function parameters, and return types. They don't enforce types at runtime but help with IDE support, documentation, and static analysis tools like mypy.

Example:

```

# Variable annotations
name: str = "Krishna"
age: int = 25
scores: list[int] = [90, 85, 88]

```

Function annotations

```
def greet(name: str, times: int = 1) -> str:  
    return f"Hello {name}! " * times
```

Complex types

```
from typing import Optional, Union, Dict, List, Tuple
```

```
def find_user(user_id: int) -> Optional[dict]:  
    """Returns user dict or None"""  
    users = {1: {"name": "Krishna"}}  
    return users.get(user_id)
```

```
def process(value: Union[int, str]) -> str:  
    return str(value)
```

Type hints for collections

```
def get_stats(numbers: List[float]) -> Dict[str, float]:  
    return {  
        "mean": sum(numbers) / len(numbers),  
        "max": max(numbers),  
        "min": min(numbers)  
    }
```

Callable type

```
from typing import Callable
```

```
def apply(func: Callable[[int, int], int], a: int, b: int) -> int:  
    return func(a, b)
```



```
result = apply(lambda x, y: x + y, 3, 5)
print(result) # 8
```

77. What is Memory Management in Python?

Answer: Python manages memory automatically using:

1. Reference counting - tracks how many references point to an object
2. Garbage collector - handles circular references
3. Memory pools - pre-allocates blocks for small objects
4. Private heap - all objects stored in a private heap managed by Python memory manager

Example:

```
import sys

# Reference counting
a = [1, 2, 3]
print(sys.getrefcount(a)) # 2 (a + function argument)

b = a # another reference
print(sys.getrefcount(a)) # 3

del b # remove reference
print(sys.getrefcount(a)) # 2

# Memory size of objects
print(sys.getsizeof(0))      # 28 bytes
print(sys.getsizeof([]))     # 56 bytes
print(sys.getsizeof("hello")) # 54 bytes
```

```

# id() shows memory address
x = 10
print(id(x))
y = 10
print(id(y)) # Same as x! (integer caching -5 to 256)

# Weak references (don't increase ref count)
import weakref

class MyObject:
    pass

obj = MyObject()
weak = weakref.ref(obj)
print(weak()) # <__main__.MyObject object>
del obj
print(weak()) # None (object was garbage collected)

```

78. What is Garbage Collection?

Answer: Garbage collection (GC) automatically reclaims memory occupied by objects that are no longer referenced. Python uses reference counting as primary mechanism and a cyclic garbage collector for circular references.

Example:

```

import gc

# Reference counting (automatic)
a = [1, 2, 3] # ref count = 1
b = a        # ref count = 2

```

```
del a      # ref count = 1
del b      # ref count = 0 -> memory freed
```

Circular reference problem

```
class Node:
```

```
    def __init__(self):
        self.ref = None
```

```
a = Node()
```

```
b = Node()
```

```
a.ref = b  # a references b
```

```
b.ref = a  # b references a (circular!)
```

```
del a
```

```
del b
```

Reference count never reaches 0, but GC handles it

Manual garbage collection

```
gc.collect() # force garbage collection
```

GC info

```
print(gc.get_count())    # (gen0, gen1, gen2) counts
```

```
print(gc.get_threshold()) # thresholds for each generation
```

Disable/Enable GC

```
gc.disable() # disable (for performance-critical code)
```

```
gc.enable() # re-enable
```

Check if GC is tracking an object

```
obj = [1, 2, 3]
```

```
print(gc.is_tracked(obj)) # True
```

79. What are Python Closures?

Answer: A closure is a function that remembers values from its enclosing scope even after the outer function has finished executing. It "closes over" the variables from the outer function.

Example:

Basic closure

```
def outer(message):  
    def inner():  
        print(message) # remembers 'message' from outer  
    return inner
```

```
hello = outer("Hello!")
```

```
hello() # Hello! (message is remembered)
```

Closure as counter

```
def make_counter():  
    count = 0  
    def increment():  
        nonlocal count  
        count += 1  
        return count  
    return increment
```

```
counter = make_counter()
```

```
print(counter()) # 1
```

```
print(counter()) # 2
```

```
print(counter()) # 3
```

```
# Closure as multiplier factory
```

```
def multiplier(factor):  
    def multiply(number):  
        return number * factor  
    return multiply
```

```
double = multiplier(2)
```

```
triple = multiplier(3)
```

```
print(double(5)) # 10
```

```
print(triple(5)) # 15
```

```
# Check closure variables
```

```
print(double.__closure__[0].cell_contents) # 2
```

```
# Practical: logging decorator using closure
```

```
def logger(prefix):  
    def decorator(func):  
        def wrapper(*args, **kwargs):  
            print(f"[{prefix}] Calling {func.__name__}")  
            return func(*args, **kwargs)  
        return wrapper  
    return decorator
```

```
@logger("DEBUG")
```

```
def add(a, b):
```

```
    return a + b
```

```
add(3, 5) # [DEBUG] Calling add
```

80. What are Python Interview Coding Challenges?

Answer: Common coding challenges include string manipulation, list operations, recursion, sorting, and algorithm problems. Here are some popular ones:

Example:

1. Reverse a string

```
def reverse_string(s):  
    return s[::-1]  
print(reverse_string("hello")) # olleh
```

2. Check palindrome

```
def is_palindrome(s):  
    s = s.lower().replace(" ", "")  
    return s == s[::-1]  
print(is_palindrome("madam")) # True
```

3. Fibonacci sequence

```
def fibonacci(n):  
    a, b = 0, 1  
    result = []  
    for _ in range(n):  
        result.append(a)  
        a, b = b, a + b  
    return result  
print(fibonacci(10)) # [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

4. Find duplicates in list

```
def find_duplicates(lst):  
    return [x for x in set(lst) if lst.count(x) > 1]
```

```
print(find_duplicates([1, 2, 2, 3, 3, 4])) # [2, 3]
```

5. Two Sum

```
def two_sum(nums, target):  
    seen = {}  
    for i, num in enumerate(nums):  
        complement = target - num  
        if complement in seen:  
            return [seen[complement], i]  
        seen[num] = i  
print(two_sum([2, 7, 11, 15], 9)) # [0, 1]
```

6. Anagram check

```
def is_anagram(s1, s2):  
    return sorted(s1.lower()) == sorted(s2.lower())  
print(is_anagram("listen", "silent")) # True
```

7. Flatten nested list

```
def flatten(lst):  
    result = []  
    for item in lst:  
        if isinstance(item, list):  
            result.extend(flatten(item))  
        else:  
            result.append(item)  
    return result  
print(flatten([1, [2, [3, 4]], 5])) # [1, 2, 3, 4, 5]
```

8. Count vowels

```
def count_vowels(s):
```

```
    return sum(1 for c in s.lower() if c in "aeiou")
print(count_vowels("Hello World")) # 3
```

9. Remove duplicates preserving order

```
def remove_duplicates(lst):
    return list(dict.fromkeys(lst))
print(remove_duplicates([1, 2, 2, 3, 1, 4])) # [1, 2, 3, 4]
```

10. FizzBuzz

```
def fizzbuzz(n):
    for i in range(1, n+1):
        if i % 15 == 0: print("FizzBuzz")
        elif i % 3 == 0: print("Fizz")
        elif i % 5 == 0: print("Buzz")
        else: print(i)
fizzbuzz(15)
```

FASTAPI INTERVIEW QUESTIONS

=====

1. What is FastAPI?

Answer: FastAPI is a modern, high-performance Python web framework for building APIs. It's built on top of Starlette (for web handling) and Pydantic (for data

validation). It supports async/await and automatically generates OpenAPI documentation.

Example:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, World!"}

@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id, "name": "Krishna"}

# Run: uvicorn main:app --reload
# Docs: http://127.0.0.1:8000/docs
```

2. Why should we use FastAPI?

Answer:

- Very fast performance (on par with NodeJS and Go)
- Automatic API documentation (Swagger UI & ReDoc)
- Type hints for auto-validation
- Async support built-in
- Easy to learn, less code
- Built-in data validation with Pydantic
- Production-ready

Example:

```
from fastapi import FastAPI

from pydantic import BaseModel


app = FastAPI(title="My API", version="1.0")


class Item(BaseModel):
    name: str
    price: float
    in_stock: bool = True


@app.post("/items/")
async def create_item(item: Item):
    # Automatic validation, docs, and serialization
    return {"item": item, "message": "Created successfully"}
```

3. FastAPI vs Flask?

Answer:

Feature	FastAPI	Flask	
-----	-----	-----	
Performance	Very fast (async)	Slower (sync)	
Async	Native support	Limited (extensions)	
Validation	Built-in (Pydantic)	Manual / extensions	
Docs	Auto-generated	Manual (Swagger ext)	
Type hints	Required & used	Optional	
Learning	Easy	Easy	
Community	Growing fast	Large, mature	

Example:

```
# Flask

from flask import Flask, request, jsonify
app = Flask(__name__)

@app.route("/items", methods=["POST"])
def create_item():
    data = request.get_json()
    # Manual validation needed
    if "name" not in data:
        return jsonify({"error": "name required"}), 400
    return jsonify(data)
```

FastAPI - same thing but better

```
from fastapi import FastAPI
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
class Item(BaseModel):
```

```
    name: str
    price: float
```

```
@app.post("/items")
```

```
async def create_item(item: Item):
```

```
    # Auto validation, auto docs, async
    return item
```

4. FastAPI vs Django?

Answer:

Feature	FastAPI	Django	
-----	-----	-----	
Type	Micro framework (API)	Full-stack framework	
Use case	APIs, microservices	Web apps, admin panels	
ORM	No built-in (use SQLAlchemy)	Built-in ORM	
Admin	No	Yes (built-in)	
Performance	Very fast	Moderate	
Async	Native	Limited (Django 4.1+)	
Templates	Not included	Built-in	

Example:

```
# Django REST - more setup
# models.py, serializers.py, views.py, urls.py needed

# FastAPI - minimal setup for same API
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class UserCreate(BaseModel):
    username: str
    email: str

@app.post("/users/")
async def create_user(user: UserCreate):
    return {"username": user.username, "email": user.email}
```

5. What are the advantages of FastAPI?

Answer:

- High performance (async, based on Starlette)
- Auto-validation with Pydantic
- Auto-generated interactive API docs
- Type safety with Python type hints
- Dependency injection system
- WebSocket support
- Easy testing with TestClient
- Standards-based (OpenAPI, JSON Schema)

Example:

```
from fastapi import FastAPI, Depends, HTTPException
from pydantic import BaseModel, EmailStr
```

```
app = FastAPI()
```

```
# Auto validation
```

```
class User(BaseModel):
```

```
    name: str
```

```
    email: EmailStr # auto validates email format
```

```
    age: int
```

```
# Dependency injection
```

```
def get_db():
```

```
    db = "database_connection"
```

```
    yield db
```

```
@app.post("/users/", response_model=User)
async def create_user(user: User, db=Depends(get_db)):
    return user # auto docs, validation, serialization all built-in
```

6. How do you install FastAPI?

Answer: Install FastAPI and an ASGI server (Uvicorn) using pip.

Example:

```
# Install FastAPI
# pip install fastapi

# Install Uvicorn (ASGI server)
# pip install uvicorn

# Or install all at once with extras
# pip install "fastapi[all]"
# This includes uvicorn, pydantic, etc.

# Create main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return {"message": "FastAPI is running!"}

# Run the server
```

```
# uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

```
# Access:
```

```
# API: http://127.0.0.1:8000
```

```
# Docs: http://127.0.0.1:8000/docs
```

```
# ReDoc: http://127.0.0.1:8000/redoc
```

7. What is Uvicorn?

Answer: Uvicorn is a lightning-fast ASGI server implementation. It runs FastAPI applications and handles HTTP requests. It's built on uvloop and httptools for maximum performance.

Example:

```
# Install
```

```
# pip install uvicorn
```

```
# Run FastAPI app
```

```
# uvicorn main:app --reload
```

```
# Common options
```

```
# uvicorn main:app --reload          (auto-reload on changes)
```

```
# uvicorn main:app --host 0.0.0.0    (accessible from network)
```

```
# uvicorn main:app --port 8080       (custom port)
```

```
# uvicorn main:app --workers 4       (multiple workers)
```

```
# Programmatic usage
```

```
import uvicorn
```

```
from fastapi import FastAPI
```

```

app = FastAPI()

@app.get("/")
def root():
    return {"status": "running"}

if __name__ == "__main__":
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)

```

8. What is ASGI?

Answer: ASGI (Asynchronous Server Gateway Interface) is the spiritual successor to WSGI. It's a specification for Python async web servers and applications. It supports HTTP, WebSocket, and other async protocols.

Example:

```

# ASGI app is a callable with scope, receive, send
async def asgi_app(scope, receive, send):
    if scope["type"] == "http":
        await send({
            "type": "http.response.start",
            "status": 200,
            "headers": [[b"content-type", b"text/plain"]],
        })
        await send({
            "type": "http.response.body",
            "body": b"Hello from ASGI!",
        })

# FastAPI is an ASGI application

```



```

from fastapi import FastAPI

app = FastAPI() # This is an ASGI app


# Run with any ASGI server:
# uvicorn, daphne, hypercorn

```

9. Difference between WSGI and ASGI?

Answer:

Feature	WSGI	ASGI	
-----	-----	-----	
Type	Synchronous	Asynchronous	
Protocol	HTTP only	HTTP, WebSocket, etc.	
Concurrency	Thread-based	Event-loop based	
Frameworks	Flask, Django	FastAPI, Starlette	
Servers	Gunicorn, uWSGI	Uvicorn, Daphne	
Performance	Good	Better for I/O-bound	

Example:

```

# WSGI app (synchronous)
def wsgi_app(environ, start_response):
    status = "200 OK"
    headers = [("Content-Type", "text/plain")]
    start_response(status, headers)
    return [b"Hello from WSGI"]


# ASGI app (asynchronous)
async def asgi_app(scope, receive, send):
    await send({

```

```

        "type": "http.response.start",
        "status": 200,
        "headers": [[b"content-type", b"text/plain"]],
    })
    await send({
        "type": "http.response.body",
        "body": b"Hello from ASGI",
    })

```

FastAPI uses ASGI -> supports async operations

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/")
```

```
async def root(): # async handler
```

```
    return {"message": "ASGI powered!"}
```

10. How do you create a FastAPI application?

Answer: Create a FastAPI instance, define route handlers with decorators, and run with Uvicorn.

Example:

```
from fastapi import FastAPI
```

```
from pydantic import BaseModel
```

```
from typing import Optional
```

```
# Create app instance
```

```
app = FastAPI(
```

```
    title="My API",
```

```
        description="A sample FastAPI application",
        version="1.0.0"
    )
```

```
# Data model
```

```
class Item(BaseModel):
    name: str
    price: float
    description: Optional[str] = None
```

```
# In-memory storage
```

```
items_db = []
```

```
# Routes
```

```
@app.get("/")
def root():
    return {"message": "Welcome to My API"}
```

```
@app.get("/items")
def get_items():
    return items_db
```

```
@app.get("/items/{item_id}")
def get_item(item_id: int):
    if item_id >= len(items_db):
        return {"error": "Item not found"}
    return items_db[item_id]
```

```
@app.post("/items")
def create_item(item: Item):
```

```
items_db.append(item.dict())  
return {"message": "Item created", "item": item}
```

```
@app.put("/items/{item_id}")  
def update_item(item_id: int, item: Item):  
    items_db[item_id] = item.dict()  
    return {"message": "Item updated"}
```

```
@app.delete("/items/{item_id}")  
def delete_item(item_id: int):  
    items_db.pop(item_id)  
    return {"message": "Item deleted"}
```

Run: uvicorn main:app --reload

11. What are Path Parameters?

Answer: Path parameters are variables in the URL path that capture values from the URL. They are defined using curly braces {} in the route path and are passed as function arguments with type hints.

Example:

```
from fastapi import FastAPI  
  
app = FastAPI()  
  
# Basic path parameter  
@app.get("/users/{user_id}")  
def get_user(user_id: int):  
    return {"user_id": user_id}  
# GET /users/42 -> {"user_id": 42}
```

```

# Multiple path parameters
@app.get("/posts/{post_id}/comments/{comment_id}")
def get_comment(post_id: int, comment_id: int):
    return {"post_id": post_id, "comment_id": comment_id}

# Path parameter with validation
from enum import Enum

class ModelName(str, Enum):
    alexnet = "alexnet"
    resnet = "resnet"
    lenet = "lenet"

@app.get("/models/{model_name}")
def get_model(model_name: ModelName):
    return {"model": model_name}

# Only accepts: /models/alexnet, /models/resnet, /models/lenet

```

12. What are Query Parameters?

Answer: Query parameters are key-value pairs in the URL after the ? mark. In FastAPI, any function parameter not in the path is automatically treated as a query parameter.

Example:

```

from fastapi import FastAPI, Query
from typing import Optional

app = FastAPI()

```

```

# Basic query parameters
@app.get("/items")
def get_items(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}
# GET /items?skip=5&limit=20

# Optional query parameter
@app.get("/search")
def search(q: Optional[str] = None, page: int = 1):
    return {"query": q, "page": page}
# GET /search?q=python&page=2

# Query parameter validation
@app.get("/items/")
def read_items(
    q: str = Query(min_length=3, max_length=50, regex="^[a-z]+$")
):
    return {"q": q}

# Required query parameter (no default)
@app.get("/users")
def get_users(role: str): # required - no default value
    return {"role": role}
# GET /users?role=admin (must provide role)

```

13. Difference between Path and Query Parameters?

Answer:

Feature	Path Parameters	Query Parameters
Location	In URL path	After ? in URL
Required	Always required	Can be optional
Syntax	/users/{id}	/users?id=1
Use case	Identify resource	Filter/sort/paginate
Example URL	/users/42	/users?skip=0&limit=10

Example:

```

from fastapi import FastAPI
from typing import Optional

app = FastAPI()

# Path param = identifies the resource
# Query param = modifies/filters the result
@app.get("/users/{user_id}/posts")
def get_user_posts(
    user_id: int,          # path - identifies which user
    skip: int = 0,         # query - pagination
    limit: int = 10,       # query - pagination
    sort_by: Optional[str] = None # query - sorting
):
    return {
        "user_id": user_id,
        "skip": skip,
        "limit": limit,
        "sort_by": sort_by
    }

# GET /users/42/posts?skip=0&limit=5&sort_by=date

```

14. What is Request Body?

Answer: Request body is data sent by the client in the body of an HTTP request (typically POST, PUT, PATCH). In FastAPI, request bodies are defined using Pydantic models for automatic validation and serialization.

Example:

```
from fastapi import FastAPI
from pydantic import BaseModel, Field
from typing import Optional

app = FastAPI()

class UserCreate(BaseModel):
    username: str = Field(min_length=3, max_length=20)
    email: str
    password: str = Field(min_length=8)
    age: Optional[int] = None

@app.post("/users/")
def create_user(user: UserCreate):
    return {"message": f"User {user.username} created", "email": user.email}

# Request body + path + query params together
class ItemUpdate(BaseModel):
    name: str
    price: float

@app.put("/items/{item_id}")
```



```
def update_item(
    item_id: int,          # path parameter
    item: ItemUpdate,      # request body
    notify: bool = False  # query parameter
):
    return {"item_id": item_id, "item": item, "notify": notify}
```

POST request body example (JSON):

```
# {
#   "username": "krishna",
#   "email": "krishna@example.com",
#   "password": "secure123",
#   "age": 25
# }
```

15. What is Pydantic?

Answer: Pydantic is a data validation library that uses Python type hints. It validates data, converts types, and provides clear error messages. FastAPI uses Pydantic models for request/response validation.

Example:

```
from pydantic import BaseModel, EmailStr, Field, validator
from typing import Optional, List
from datetime import datetime
```

```
class Address(BaseModel):
    street: str
    city: str
    zipcode: str
```

```

class User(BaseModel):
    name: str = Field(min_length=2, max_length=50)
    email: EmailStr
    age: int = Field(gt=0, lt=150)
    address: Optional[Address] = None
    tags: List[str] = []
    created_at: datetime = Field(default_factory=datetime.now)

    @validator("name")
    def name_must_be_alpha(cls, v):
        if not v.replace(" ", "").isalpha():
            raise ValueError("Name must contain only letters")
        return v.title()

# Valid data
user = User(name="krishna", email="k@example.com", age=25)
print(user.name)    # Krishna (title case from validator)
print(user.dict())  # converts to dictionary
print(user.json())  # converts to JSON string

# Invalid data raises ValidationError
try:
    bad_user = User(name="", email="invalid", age=-1)
except Exception as e:
    print(e) # shows detailed validation errors

```

16. Why is Pydantic used in FastAPI?

Answer: Pydantic provides:

- Automatic request body validation
- Type coercion (converts compatible types)
- Clear error messages for invalid data
- JSON serialization/deserialization
- Response model filtering
- Schema generation for OpenAPI docs

Example:

```
from fastapi import FastAPI
from pydantic import BaseModel, EmailStr

app = FastAPI()

class UserIn(BaseModel):
    username: str
    email: EmailStr
    password: str

class UserOut(BaseModel):
    username: str
    email: EmailStr
    # password excluded from response!

@app.post("/users/", response_model=UserOut)
def create_user(user: UserIn):
    # Even though we return full user, response_model filters out password
    return user

# If client sends invalid data:
```

```
# POST {"username": "k", "email": "not-email", "password": "short"}
```

```
# Response: 422 Unprocessable Entity with detailed errors
```

17. What are Pydantic Models?

Answer: Pydantic models are classes that inherit from BaseModel. They define the shape and validation rules for data. Each field has a type annotation and optional validators.

Example:

```
from pydantic import BaseModel, Field
```

```
from typing import Optional, List
```

```
from datetime import datetime
```

```
from enum import Enum
```

```
class Priority(str, Enum):
```

```
    low = "low"
```

```
    medium = "medium"
```

```
    high = "high"
```

```
class TaskBase(BaseModel):
```

```
    title: str = Field(min_length=1, max_length=100)
```

```
    description: Optional[str] = None
```

```
    priority: Priority = Priority.medium
```

```
class TaskCreate(TaskBase):
```

```
    pass
```

```
class TaskResponse(TaskBase):
```

```
    id: int
```

```

created_at: datetime
completed: bool = False

class Config:
    from_attributes = True # allows ORM model conversion

# Nested models
class Project(BaseModel):
    name: str
    tasks: List[TaskResponse] = []
    owner: str

# Usage
task = TaskCreate(title="Build API", priority="high")
print(task.model_dump()) # {'title': 'Build API', 'description': None, 'priority': 'high'}

```

18. How do you validate request data?

Answer: FastAPI validates request data automatically using Pydantic models, Path/Query parameters with constraints, and custom validators.

Example:

```

from fastapi import FastAPI, Path, Query, Body, HTTPException
from pydantic import BaseModel, Field, validator

app = FastAPI()

class Product(BaseModel):
    name: str = Field(min_length=1, max_length=100)
    price: float = Field(gt=0, description="Price must be positive")

```

```
quantity: int = Field(ge=0)
```

```
@validator("name")
```

```
def name_not_empty(cls, v):
```

```
    if v.strip() == "":
```

```
        raise ValueError("Name cannot be empty/whitespace")
```

```
    return v
```

```
@app.post("/products/")
```

```
def create_product(product: Product):
```

```
    return product
```

```
# Path parameter validation
```

```
@app.get("/items/{item_id}")
```

```
def get_item(
```

```
    item_id: int = Path(gt=0, le=1000, description="Item ID must be 1-1000")
```

```
):
```

```
    return {"item_id": item_id}
```

```
# Query parameter validation
```

```
@app.get("/search")
```

```
def search(
```

```
    q: str = Query(min_length=3, max_length=50),
```

```
    page: int = Query(default=1, ge=1)
```

```
):
```

```
    return {"q": q, "page": page}
```

```
# Invalid request auto-returns 422 with error details
```

19. What are Response Models?

Answer: Response models define the shape of the API response. They filter out unwanted fields, validate output data, and generate response schema in docs. Set via `response_model` parameter.

Example:

```
from fastapi import FastAPI

from pydantic import BaseModel, EmailStr
from typing import List

app = FastAPI()

class UserDB(BaseModel):
    id: int
    username: str
    email: EmailStr
    hashed_password: str # stored in DB

class UserResponse(BaseModel):
    id: int
    username: str
    email: EmailStr
    # No password field - excluded from response!

fake_db = [
    UserDB(id=1, username="krishna", email="k@test.com",
hashed_password="hash123")
]

@app.get("/users/", response_model=List[UserResponse])
```

```

def get_users():
    return fake_db # password automatically excluded

@app.get("/users/{user_id}", response_model=UserResponse)
def get_user(user_id: int):
    return fake_db[0] # hashed_password filtered out

# response_model_exclude

@app.get("/users-v2/{user_id}", response_model=UserDB,
response_model_exclude={"hashed_password"})
def get_user_v2(user_id: int):
    return fake_db[0]

```

20. How do you return custom responses?

Answer: FastAPI provides several response classes for different content types: JSONResponse, HTMLResponse, PlainTextResponse, RedirectResponse, StreamingResponse, FileResponse.

Example:

```

from fastapi import FastAPI
from fastapi.responses import (
    JSONResponse, HTMLResponse, PlainTextResponse,
    RedirectResponse, FileResponse, StreamingResponse
)

app = FastAPI()

# Custom JSON response with status code
@app.get("/custom")

```



```
def custom_response():
    return JsonResponse(
        content={"message": "Custom response"},
        status_code=201,
        headers={"X-Custom-Header": "value"}
    )

# HTML response
@app.get("/page", response_class=HTMLResponse)
def get_page():
    return "<h1>Hello World</h1><p>This is HTML</p>"

# Plain text
@app.get("/text", response_class=PlainTextResponse)
def get_text():
    return "Plain text response"

# Redirect
@app.get("/old-page")
def redirect():
    return RedirectResponse(url="/new-page")

# File download
@app.get("/download")
def download_file():
    return FileResponse("report.pdf", filename="report.pdf")

# Streaming response
@app.get("/stream")
def stream():
```

```
def generate():
    for i in range(10):
        yield f"chunk {i}\n"
    return StreamingResponse(generate(), media_type="text/plain")
```

21. What are HTTP Methods?

Answer: HTTP methods define the action to perform on a resource:

- GET: Retrieve data
- POST: Create new resource
- PUT: Update/replace entire resource
- PATCH: Partially update resource
- DELETE: Remove resource
- OPTIONS: Get supported methods
- HEAD: Get headers only

Example:

```
from fastapi import FastAPI
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
class Item(BaseModel):
    name: str
    price: float
```

```
items = {}
```

```
@app.get("/items/{item_id}")
```

```
def read_item(item_id: int):
```

```
return items.get(item_id)
```

```
@app.post("/items/")
```

```
def create_item(item: Item):
```

```
    item_id = len(items) + 1
```

```
    items[item_id] = item
```

```
    return {"id": item_id, "item": item}
```

```
@app.put("/items/{item_id}")
```

```
def replace_item(item_id: int, item: Item):
```

```
    items[item_id] = item
```

```
    return {"id": item_id, "item": item}
```

```
@app.patch("/items/{item_id}")
```

```
def update_item(item_id: int, name: str = None):
```

```
    if name:
```

```
        items[item_id].name = name
```

```
    return items[item_id]
```

```
@app.delete("/items/{item_id}")
```

```
def delete_item(item_id: int):
```

```
    del items[item_id]
```

```
    return {"message": "Deleted"}
```

22. Difference between GET and POST?

Answer:

Feature	GET	POST	
-----	-----	-----	

Purpose	Retrieve data	Create/submit data	
Body	No request body	Has request body	
Cacheable	Yes	No	
Idempotent	Yes	No	
Bookmarkable	Yes	No	
Data in	URL (query params)	Request body	
Security	Less (data visible in URL)	More (data in body)	

Example:

```

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class LoginData(BaseModel):
    username: str
    password: str

# GET - retrieve data, params in URL
@app.get("/users")
def get_users(skip: int = 0, limit: int = 10):
    # GET /users?skip=0&limit=10
    return {"skip": skip, "limit": limit}

# POST - send data in body, creates resource
@app.post("/login")
def login(data: LoginData):
    # Password in body, not visible in URL
    return {"message": f"Welcome {data.username}"}

```

23. Difference between PUT and PATCH?

Answer:

Feature	PUT	PATCH	
-----	-----	-----	
Update	Full replacement	Partial update	
Body	Complete resource	Only changed fields	
Idempotent	Yes	Not necessarily	
Missing	Resets to default	Unchanged	

Example:

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import Optional
```

```
app = FastAPI()
```

```
class Item(BaseModel):
```

```
    name: str
```

```
    price: float
```

```
    description: str = ""
```

```
class ItemPatch(BaseModel):
```

```
    name: Optional[str] = None
```

```
    price: Optional[float] = None
```

```
    description: Optional[str] = None
```

```
db = {1: Item(name="Phone", price=999, description="Smartphone")}
```

```

# PUT - replaces entire resource
@app.put("/items/{item_id}")
def replace_item(item_id: int, item: Item):
    db[item_id] = item # complete replacement
    return db[item_id]

# Must send ALL fields: {"name": "Phone", "price": 899, "description": "New
phone"}

# PATCH - updates only provided fields
@app.patch("/items/{item_id}")
def patch_item(item_id: int, item: ItemPatch):
    stored = db[item_id]
    update_data = item.dict(exclude_unset=True)
    updated = stored.copy(update=update_data)
    db[item_id] = updated
    return updated

# Can send just: {"price": 899} (other fields unchanged)

```

24. Difference between PUT and DELETE?

Answer:

Feature	PUT	DELETE
Purpose	Update/replace resource	Remove resource
Body	Has request body	Usually no body
Idempotent	Yes	Yes
Response	Updated resource	Confirmation message

Example:

```

from fastapi import FastAPI, HTTPException

```

```

from pydantic import BaseModel

app = FastAPI()

class Task(BaseModel):
    title: str
    done: bool = False

tasks = {1: Task(title="Learn FastAPI", done=False)}

# PUT - update the resource
@app.put("/tasks/{task_id}")
def update_task(task_id: int, task: Task):
    if task_id not in tasks:
        raise HTTPException(status_code=404, detail="Task not found")
    tasks[task_id] = task
    return {"task_id": task_id, "task": task}

# DELETE - remove the resource
@app.delete("/tasks/{task_id}")
def delete_task(task_id: int):
    if task_id not in tasks:
        raise HTTPException(status_code=404, detail="Task not found")
    del tasks[task_id]
    return {"message": f"Task {task_id} deleted"}

```

25. What is Dependency Injection in FastAPI?

Answer: Dependency Injection (DI) is a design pattern where dependencies (database connections, auth, configs) are injected into functions rather than created inside them. FastAPI has a built-in DI system using Depends().

Example:

```
from fastapi import FastAPI, Depends, HTTPException, Header
```

```
app = FastAPI()
```

```
# Simple dependency
```

```
def get_db():
```

```
    db = {"connection": "active"}
```

```
    try:
```

```
        yield db
```

```
    finally:
```

```
        print("Closing DB connection")
```

```
# Auth dependency
```

```
def get_current_user(token: str = Header()):
```

```
    if token != "valid-token":
```

```
        raise HTTPException(status_code=401, detail="Invalid token")
```

```
    return {"user": "krishna", "role": "admin"}
```

```
@app.get("/items/")
```

```
def get_items(db=Depends(get_db), user=Depends(get_current_user)):
```

```
    return {"db": db, "user": user}
```

```
# Dependency with parameters
```

```
def pagination(skip: int = 0, limit: int = 10):
```

```
    return {"skip": skip, "limit": limit}
```



```
@app.get("/users/")
def get_users(params=Depends(pagination)):
    return params
```

26. What is Depends()?

Answer: Depends() is FastAPI's function for declaring dependencies. It takes a callable (function/class) and injects its return value into the route handler. Dependencies can be nested and reused across routes.

Example:

```
from fastapi import FastAPI, Depends, HTTPException
```

```
app = FastAPI()
```

```
# Function dependency
```

```
def verify_api_key(api_key: str):
    if api_key != "secret123":
        raise HTTPException(status_code=403, detail="Invalid API key")
    return api_key
```

```
@app.get("/protected")
```

```
def protected_route(key=Depends(verify_api_key)):
    return {"message": "Access granted", "key": key}
```

```
# Class dependency
```

```
class CommonParams:
```

```
    def __init__(self, skip: int = 0, limit: int = 100, q: str = None):
        self.skip = skip
        self.limit = limit
```

```
self.q = q
```

```
@app.get("/items/")
```

```
def get_items(params: CommonParams = Depends()):
```

```
    return {"skip": params.skip, "limit": params.limit, "q": params.q}
```

```
# Nested dependencies
```

```
def get_db():
```

```
    return "db_connection"
```

```
def get_repo(db=Depends(get_db)):
```

```
    return {"repo": "user_repo", "db": db}
```

```
@app.get("/users/")
```

```
def get_users(repo=Depends(get_repo)):
```

```
    return repo
```

27. How do you create reusable dependencies?

Answer: Create dependencies as separate functions or classes and use them across multiple routes. You can also apply dependencies at router or app level.

Example:

```
from fastapi import FastAPI, Depends, HTTPException, APIRouter
```

```
app = FastAPI()
```

```
# Reusable dependency
```

```
def pagination(skip: int = 0, limit: int = 10):
```

```
    if limit > 100:
```

```

        raise HTTPException(400, "Limit cannot exceed 100")
    return {"skip": skip, "limit": limit}

# Reusable auth dependency
def require_admin(role: str = "user"):
    if role != "admin":
        raise HTTPException(403, "Admin access required")
    return True

# Use same dependency in multiple routes
@app.get("/users/")
def get_users(page=Depends(pagination)):
    return {"users": [], **page}

@app.get("/posts/")
def get_posts(page=Depends(pagination)):
    return {"posts": [], **page}

# Router-level dependency (applies to all routes in router)
admin_router = APIRouter(
    prefix="/admin",
    dependencies=[Depends(require_admin)]
)

@admin_router.get("/dashboard")
def admin_dashboard():
    return {"message": "Admin dashboard"}

app.include_router(admin_router)

```

```
# App-level dependency (applies to ALL routes)
# app = FastAPI(dependencies=[Depends(verify_api_key)])
```

28. What is Middleware?

Answer: Middleware is code that runs before and after every request. It can modify requests/responses, add headers, log requests, measure performance, handle CORS, etc. Middleware wraps the entire request-response cycle.

Example:

```
from fastapi import FastAPI, Request
import time

app = FastAPI()

# Custom middleware
@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    start_time = time.time()
    response = await call_next(request)
    process_time = time.time() - start_time
    response.headers["X-Process-Time"] = str(process_time)
    return response

# Logging middleware
@app.middleware("http")
async def log_requests(request: Request, call_next):
    print(f"Request: {request.method} {request.url}")
    response = await call_next(request)
    print(f"Response: {response.status_code}")
```

```
return response
```

```
@app.get("/")
```

```
def root():
```

```
    return {"message": "Hello"}
```

29. How do you add Middleware in FastAPI?

Answer: Use `@app.middleware("http")` decorator or `app.add_middleware()` for class-based middleware.

Example:

```
from fastapi import FastAPI, Request
```

```
from fastapi.middleware.cors import CORSMiddleware
```

```
from starlette.middleware.base import BaseHTTPMiddleware
```

```
app = FastAPI()
```

```
# Method 1: Decorator-based
```

```
@app.middleware("http")
```

```
async def custom_middleware(request: Request, call_next):
```

```
    # Before request
```

```
    print(f"Incoming: {request.method} {request.url}")
```

```
    response = await call_next(request)
```

```
    # After request
```

```
    response.headers["X-App-Version"] = "1.0"
```

```
    return response
```

```
# Method 2: Class-based middleware
```

```
class AuthMiddleware(BaseHTTPMiddleware):
```

```

async def dispatch(self, request: Request, call_next):
    if request.url.path.startswith("/api"):
        token = request.headers.get("Authorization")
        if not token:
            from fastapi.responses import JSONResponse
            return JSONResponse(
                status_code=401,
                content={"detail": "Not authenticated"}
            )
        response = await call_next(request)
        return response

```

```
app.add_middleware(AuthMiddleware)
```

Method 3: Built-in middleware

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

```

30. What is CORS?

Answer: CORS (Cross-Origin Resource Sharing) is a security mechanism that restricts web pages from making requests to a different domain. Browsers block cross-origin requests by default. CORS headers tell browsers which origins are allowed.

Example:

```

from fastapi import FastAPI

from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

# Allow specific origins
origins = [
    "http://localhost:3000",    # React dev server
    "https://myapp.com",       # Production frontend
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,      # which origins can access
    allow_credentials=True,     # allow cookies
    allow_methods=["GET", "POST", "PUT", "DELETE"], # allowed HTTP methods
    allow_headers=["*"],        # allowed headers
    max_age=600,               # cache preflight for 10 min
)

# Allow all origins (development only!)
# app.add_middleware(
#     CORSMiddleware,
#     allow_origins=["*"],
#     allow_methods=["*"],
#     allow_headers=["*"],
# )

@app.get("/api/data")
def get_data():

```

```
return {"message": "This can be accessed cross-origin"}
```

31. How do you enable CORS in FastAPI?

Answer: Add CORSMiddleware from `fastapi.middleware.cors` with appropriate configuration for allowed origins, methods, and headers.

Example:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000", "https://production.com"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["Authorization", "Content-Type"],
    expose_headers=["X-Custom-Header"],
    max_age=3600,
)

@app.get("/")
def root():
    return {"message": "CORS enabled"}
```

32. What is Authentication?

Answer: Authentication verifies WHO the user is (identity). It confirms that users are who they claim to be using credentials like username/password, tokens, or API keys.

Example:

```
from fastapi import FastAPI, Depends, HTTPException
from fastapi.security import HTTPBasic, HTTPBasicCredentials
import secrets

app = FastAPI()
security = HTTPBasic()

# Basic Authentication
def authenticate(credentials: HTTPBasicCredentials = Depends(security)):
    correct_user = secrets.compare_digest(credentials.username, "admin")
    correct_pass = secrets.compare_digest(credentials.password, "secret")
    if not (correct_user and correct_pass):
        raise HTTPException(status_code=401, detail="Invalid credentials")
    return credentials.username

@app.get("/protected")
def protected_route(username: str = Depends(authenticate)):
    return {"message": f"Hello {username}, you are authenticated!"}

# Token-based authentication
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials

bearer = HTTPBearer()

def verify_token(credentials: HTTPAuthorizationCredentials = Depends(bearer)):
    if credentials.credentials != "valid-token-123":
        raise HTTPException(status_code=401, detail="Invalid token")
```

```
return {"user": "krishna"}
```

```
@app.get("/api/data")
```

```
def get_data(user=Depends(verify_token)):
```

```
    return {"data": "secret", "user": user}
```

33. What is Authorization?

Answer: Authorization determines WHAT a user can do (permissions). After authentication confirms identity, authorization checks if the user has permission to perform the requested action.

Example:

```
from fastapi import FastAPI, Depends, HTTPException
```

```
app = FastAPI()
```

```
# Simulated user database
```

```
users_db = {
```

```
    "krishna": {"role": "admin", "permissions": ["read", "write", "delete"]},
```

```
    "ravi": {"role": "user", "permissions": ["read"]},
```

```
}
```

```
def get_current_user(username: str):
```

```
    if username not in users_db:
```

```
        raise HTTPException(401, "User not found")
```

```
    return users_db[username]
```

```
# Role-based authorization
```

```
def require_role(required_role: str):
```

```

def checker(username: str):
    user = get_current_user(username)
    if user["role"] != required_role:
        raise HTTPException(403, f'Requires {required_role} role')
    return user
return checker

# Permission-based authorization
def require_permission(permission: str):
    def checker(username: str):
        user = get_current_user(username)
        if permission not in user["permissions"]:
            raise HTTPException(403, f'Missing permission: {permission}')
        return user
    return checker

@app.get("/admin")
def admin_panel(user=Depends(require_role("admin"))):
    return {"message": "Admin panel", "user": user}

@app.delete("/items/{item_id}")
def delete_item(item_id: int, user=Depends(require_permission("delete"))):
    return {"message": f'Item {item_id} deleted by {user["role"]}'}

```

34. How do you implement JWT Authentication?

Answer: JWT (JSON Web Token) authentication involves creating tokens on login and verifying them on protected routes. Use python-jose for JWT encoding/decoding.

Example:

```

from fastapi import FastAPI, Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from jose import JWTError, jwt
from datetime import datetime, timedelta
from pydantic import BaseModel

app = FastAPI()

SECRET_KEY = "your-secret-key"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

# Create JWT token
def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() +
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

# Verify JWT token
def get_current_user(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username = payload.get("sub")
        if username is None:
            raise HTTPException(401, "Invalid token")
        return {"username": username}

```

```

except JWTError:
    raise HTTPException(401, "Invalid token")

# Login endpoint
@app.post("/token")
def login(form_data: OAuth2PasswordRequestForm = Depends()):
    # Verify credentials (simplified)
    if form_data.username == "admin" and form_data.password == "password":
        token = create_access_token(data={"sub": form_data.username})
        return {"access_token": token, "token_type": "bearer"}
    raise HTTPException(401, "Invalid credentials")

# Protected route
@app.get("/users/me")
def read_users_me(user=Depends(get_current_user)):
    return user

```

35. What is OAuth2?

Answer: OAuth2 is an authorization framework that allows third-party applications to access user resources without exposing credentials. It uses access tokens and supports flows like Authorization Code, Client Credentials, and Password.

Example:

```

from fastapi import FastAPI, Depends

from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm

app = FastAPI()

```

```

# OAuth2 password flow
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

fake_users = {
    "krishna": {"username": "krishna", "password": "pass123", "role": "admin"}
}

@app.post("/token")
def login(form_data: OAuth2PasswordRequestForm = Depends()):
    user = fake_users.get(form_data.username)
    if not user or user["password"] != form_data.password:
        from fastapi import HTTPException
        raise HTTPException(400, "Incorrect username or password")
    return {"access_token": user["username"], "token_type": "bearer"}

@app.get("/users/me")
def get_me(token: str = Depends(oauth2_scheme)):
    return {"user": token}

# OAuth2 scopes for fine-grained permissions
from fastapi.security import SecurityScopes

# Scopes: read:users, write:users, admin
# Token includes scopes, and routes check required scopes

```

36. How do you secure FastAPI APIs?

Answer: Secure APIs using JWT auth, HTTPS, rate limiting, input validation, CORS, and security headers.

Example:

```
from fastapi import FastAPI, Depends, HTTPException, Request
from fastapi.security import OAuth2PasswordBearer
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.trustedhost import TrustedHostMiddleware
import time

app = FastAPI()

# 1. CORS
app.add_middleware(CORSMiddleware, allow_origins=["https://myapp.com"])

# 2. Trusted hosts
app.add_middleware(TrustedHostMiddleware, allowed_hosts=["myapp.com",
"*.myapp.com"])

# 3. Rate limiting middleware
request_counts = {}

@app.middleware("http")
async def rate_limit(request: Request, call_next):
    client_ip = request.client.host
    now = time.time()
    if client_ip in request_counts:
        count, timestamp = request_counts[client_ip]
        if now - timestamp < 60 and count >= 100:
            from fastapi.responses import JSONResponse
            return JSONResponse(status_code=429, content={"detail": "Too many
requests"})
        if now - timestamp >= 60:
            request_counts[client_ip] = (1, now)
```

```

    else:
        request_counts[client_ip] = (count + 1, timestamp)
    else:
        request_counts[client_ip] = (1, now)
    return await call_next(request)

```

4. Security headers

```

@app.middleware("http")
async def add_security_headers(request: Request, call_next):
    response = await call_next(request)
    response.headers["X-Content-Type-Options"] = "nosniff"
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["Strict-Transport-Security"] = "max-age=31536000"
    return response

```

5. Input validation (automatic with Pydantic)

```

from pydantic import BaseModel, Field

```

```

class SecureInput(BaseModel):
    data: str = Field(max_length=1000)

```

37. How do you handle Exceptions?

Answer: Use HTTPException for HTTP errors, custom exception handlers for specific exceptions, and global handlers for unhandled errors.

Example:

```

from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import JSONResponse

```



```
app = FastAPI()
```

```
# Built-in HTTPException
```

```
@app.get("/items/{item_id}")
```

```
def get_item(item_id: int):
```

```
    if item_id not in [1, 2, 3]:
```

```
        raise HTTPException(
```

```
            status_code=404,
```

```
            detail="Item not found",
```

```
            headers={"X-Error": "Item missing"}  
        )
```

```
    return {"item_id": item_id}
```

```
# Custom exception class
```

```
class ItemNotFoundError(Exception):
```

```
    def __init__(self, item_id: int):
```

```
        self.item_id = item_id
```

```
# Register exception handler
```

```
@app.exception_handler(ItemNotFoundError)
```

```
async def item_not_found_handler(request: Request, exc: ItemNotFoundError):
```

```
    return JSONResponse(
```

```
        status_code=404,
```

```
        content={"error": f"Item {exc.item_id} not found", "type": "ItemNotFound"}  
    )
```

```
@app.get("/products/{product_id}")
```

```
def get_product(product_id: int):
```

```
    if product_id > 100:
```

```
        raise ItemNotFoundError(product_id)
```

```

        return {"product_id": product_id}

# Global exception handler
@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    return JSONResponse(status_code=500, content={"error": "Internal server error"})

```

38. How do you create Custom Exception Handlers?

Answer: Define custom exception classes and register handlers using `@app.exception_handler()` decorator.

Example:

```

from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

app = FastAPI()

# Custom exceptions
class AppException(Exception):
    def __init__(self, status_code: int, detail: str, error_code: str):
        self.status_code = status_code
        self.detail = detail
        self.error_code = error_code

class DatabaseError(AppException):
    def __init__(self, detail: str = "Database error"):
        super().__init__(500, detail, "DB_ERROR")

```

```

class NotFoundError(AppException):
    def __init__(self, resource: str):
        super().__init__(404, f"{resource} not found", "NOT_FOUND")

class ValidationError(AppException):
    def __init__(self, detail: str):
        super().__init__(422, detail, "VALIDATION_ERROR")

# Register handler for base exception
@app.exception_handler(AppException)
async def app_exception_handler(request: Request, exc: AppException):
    return JSONResponse(
        status_code=exc.status_code,
        content={
            "error": exc.detail,
            "error_code": exc.error_code,
            "path": str(request.url)
        }
    )

@app.get("/users/{user_id}")
def get_user(user_id: int):
    if user_id > 100:
        raise NotFoundError("User")
    return {"user_id": user_id}

```

39. What are Background Tasks?

Answer: Background tasks run after the response is sent to the client. They're useful for operations that don't need to block the response, like sending emails, logging, or processing data.

Example:

```
from fastapi import FastAPI, BackgroundTasks

app = FastAPI()

# Background task function
def send_email(email: str, message: str):
    import time
    time.sleep(3) # simulate sending email
    print(f"Email sent to {email}: {message}")

def write_log(message: str):
    with open("log.txt", "a") as f:
        f.write(f"{message}\n")

@app.post("/register")
def register_user(email: str, background_tasks: BackgroundTasks):
    # Response sent immediately
    background_tasks.add_task(send_email, email, "Welcome!")
    background_tasks.add_task(write_log, f"User {email} registered")
    return {"message": "User registered", "email": email}
    # Email sent AFTER response
```

40. How do Background Tasks work?

Answer: BackgroundTasks uses Starlette's background task system. Tasks are added to a queue and executed after the response is returned. They run in the same process (not a separate worker like Celery).

Example:

```
from fastapi import FastAPI, BackgroundTasks, Depends

app = FastAPI()

# Multiple background tasks
def process_image(image_id: int):
    import time
    time.sleep(5)
    print(f"Image {image_id} processed")

def notify_user(user_id: int, message: str):
    print(f"Notifying user {user_id}: {message}")

def update_analytics(action: str):
    print(f"Analytics updated: {action}")

@app.post("/upload/{image_id}")
async def upload_image(image_id: int, background_tasks: BackgroundTasks):
    # All these run after response is sent
    background_tasks.add_task(process_image, image_id)
    background_tasks.add_task(notify_user, 1, "Image uploaded")
    background_tasks.add_task(update_analytics, "image_upload")
    return {"status": "Upload started", "image_id": image_id}

# Background tasks in dependencies
def log_dependency(background_tasks: BackgroundTasks):
```

```
background_tasks.add_task(update_analytics, "api_call")
```

```
@app.get("/items/", dependencies=[Depends(log_dependency)])
```

```
def get_items():
```

```
    return {"items": []}
```

NOTE: For heavy tasks, use Celery or similar task queue instead

41. What are WebSockets in FastAPI?

Answer: WebSockets provide full-duplex, bidirectional communication between client and server over a single TCP connection. Unlike HTTP (request-response), WebSockets allow real-time data streaming (chat apps, live updates, notifications).

Example:

```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
```

```
app = FastAPI()
```

```
@app.websocket("/ws")
```

```
async def websocket_endpoint(ws: WebSocket):
```

```
    await ws.accept()
```

```
    try:
```

```
        while True:
```

```
            data = await ws.receive_text()
```

```
            await ws.send_text(f"Echo: {data}")
```

```
    except WebSocketDisconnect:
```

```
        print("Client disconnected")
```

```
# Client (JavaScript):
```

```
# const ws = new WebSocket("ws://localhost:8000/ws");
```

```
# ws.onmessage = (event) => console.log(event.data);  
# ws.send("Hello!");
```

42. How do you implement WebSockets?

Answer: Use `@app.websocket()` decorator, accept connections, and handle send/receive in a loop. Use a connection manager for multiple clients.

Example:

```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect  
from typing import List
```

```
app = FastAPI()
```

```
# Connection manager for chat
```

```
class ConnectionManager:
```

```
    def __init__(self):
```

```
        self.active_connections: List[WebSocket] = []
```

```
    async def connect(self, websocket: WebSocket):
```

```
        await websocket.accept()
```

```
        self.active_connections.append(websocket)
```

```
    def disconnect(self, websocket: WebSocket):
```

```
        self.active_connections.remove(websocket)
```

```
    async def broadcast(self, message: str):
```

```
        for connection in self.active_connections:
```

```
            await connection.send_text(message)
```

```
manager = ConnectionManager()
```

```
@app.websocket("/ws/{username}")
```

```
async def chat(websocket: WebSocket, username: str):
```

```
    await manager.connect(websocket)
```

```
    await manager.broadcast(f"{username} joined the chat")
```

```
    try:
```

```
        while True:
```

```
            data = await websocket.receive_text()
```

```
            await manager.broadcast(f"{username}: {data}")
```

```
    except WebSocketDisconnect:
```

```
        manager.disconnect(websocket)
```

```
        await manager.broadcast(f"{username} left the chat")
```

43. What is File Upload in FastAPI?

Answer: FastAPI handles file uploads using UploadFile and File from fastapi. UploadFile uses spooled temporary files and provides async read/write methods.

Example:

```
from fastapi import FastAPI, UploadFile, File
```

```
app = FastAPI()
```

```
@app.post("/upload/")
```

```
async def upload_file(file: UploadFile = File(...)):
```

```
    contents = await file.read()
```

```
    # Save file
```

```
    with open(f"uploads/{file.filename}", "wb") as f:
```

```
        f.write(contents)
```



```

return {
    "filename": file.filename,
    "size": len(contents),
    "content_type": file.content_type
}

# With file size validation
@app.post("/upload-image/")
async def upload_image(file: UploadFile = File(...)):
    if file.content_type not in ["image/png", "image/jpeg"]:
        from fastapi import HTTPException
        raise HTTPException(400, "Only PNG/JPEG allowed")

    contents = await file.read()
    if len(contents) > 5 * 1024 * 1024: # 5MB limit
        raise HTTPException(400, "File too large (max 5MB)")

    with open(f"uploads/{file.filename}", "wb") as f:
        f.write(contents)
    return {"filename": file.filename}

```

44. How do you upload multiple files?

Answer: Use List[UploadFile] to accept multiple files in a single request.

Example:

```

from fastapi import FastAPI, UploadFile, File
from typing import List

```

```

app = FastAPI()

@app.post("/upload-multiple/")
async def upload_files(files: List[UploadFile] = File(...)):
    results = []
    for file in files:
        contents = await file.read()
        with open(f'uploads/{file.filename}', "wb") as f:
            f.write(contents)
        results.append({
            "filename": file.filename,
            "size": len(contents)
        })
    return {"uploaded": len(results), "files": results}

```

With additional form data

```
from fastapi import Form
```

```

@app.post("/upload-with-data/")
async def upload_with_data(
    description: str = Form(...),
    files: List[UploadFile] = File(...)
):
    filenames = [f.filename for f in files]
    return {"description": description, "files": filenames}

```

45. How do you connect FastAPI with a Database?

Answer: Use SQLAlchemy (ORM) or databases (async) library. Create a database session dependency and inject it into routes.

Example:

```
from fastapi import FastAPI, Depends
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session

# Database setup
DATABASE_URL = "sqlite:///./app.db"

engine = create_engine(DATABASE_URL, connect_args={"check_same_thread":
False})

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

# Model
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, index=True)
    email = Column(String, unique=True)

Base.metadata.create_all(bind=engine)

# Dependency
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

```
app = FastAPI()
```

```
@app.post("/users/")
```

```
def create_user(name: str, email: str, db: Session = Depends(get_db)):
```

```
    user = User(name=name, email=email)
```

```
    db.add(user)
```

```
    db.commit()
```

```
    db.refresh(user)
```

```
    return {"id": user.id, "name": user.name, "email": user.email}
```

```
@app.get("/users/")
```

```
def get_users(db: Session = Depends(get_db)):
```

```
    return db.query(User).all()
```

46. FastAPI with SQLAlchemy?

Answer: SQLAlchemy is the most popular ORM for Python. Use it with FastAPI by creating models, schemas, and a database dependency.

Example:

```
# --- database.py ---
```

```
from sqlalchemy import create_engine
```

```
from sqlalchemy.ext.declarative import declarative_base
```

```
from sqlalchemy.orm import sessionmaker
```

```
SQLALCHEMY_DATABASE_URL = "sqlite:///./sql_app.db"
```

```
engine = create_engine(SQLALCHEMY_DATABASE_URL)
```

```
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

```
Base = declarative_base()
```

```

# --- models.py ---
from sqlalchemy import Column, Integer, String, Float, Boolean

class Product(Base):
    __tablename__ = "products"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, index=True)
    price = Column(Float)
    in_stock = Column(Boolean, default=True)

# --- schemas.py ---
from pydantic import BaseModel

class ProductCreate(BaseModel):
    name: str
    price: float
    in_stock: bool = True

class ProductResponse(ProductCreate):
    id: int

    class Config:
        from_attributes = True

# --- main.py ---
from fastapi import FastAPI, Depends
from sqlalchemy.orm import Session

app = FastAPI()
Base.metadata.create_all(bind=engine)

```

```

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.post("/products/", response_model=ProductResponse)
def create_product(product: ProductCreate, db: Session = Depends(get_db)):
    db_product = Product(**product.dict())
    db.add(db_product)
    db.commit()
    db.refresh(db_product)
    return db_product

```

47. FastAPI with PostgreSQL?

Answer: Use asyncpg for async or psycopg2 for sync PostgreSQL connections with SQLAlchemy.

Example:

```

# Install: pip install sqlalchemy psycopg2-binary
# For async: pip install asyncpg sqlalchemy[asyncio]

```

```

# Sync PostgreSQL

```

```

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

```

```

DATABASE_URL = "postgresql://user:password@localhost:5432/mydb"
engine = create_engine(DATABASE_URL)

```

```

SessionLocal = sessionmaker(bind=engine)

# Async PostgreSQL
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker

ASYNC_DATABASE_URL =
"postgresql+asyncpg://user:password@localhost:5432/mydb"
async_engine = create_async_engine(ASYNC_DATABASE_URL)

AsyncSessionLocal = sessionmaker(async_engine, class_=AsyncSession,
expire_on_commit=False)

async def get_async_db():
    async with AsyncSessionLocal() as session:
        yield session

# Usage in route
from fastapi import FastAPI, Depends

app = FastAPI()

@app.get("/users/")
async def get_users(db: AsyncSession = Depends(get_async_db)):
    from sqlalchemy import select
    result = await db.execute(select(User))
    return result.scalars().all()

```

48. What is Alembic?

Answer: Alembic is a database migration tool for SQLAlchemy. It tracks database schema changes (like Git for databases) and allows you to upgrade/downgrade database versions.

Example:

```
# Install: pip install alembic
```

```
# Initialize
```

```
# alembic init alembic
```

```
# Configure alembic.ini:
```

```
# sqlalchemy.url = postgresql://user:pass@localhost/mydb
```

```
# Edit alembic/env.py:
```

```
# from models import Base
```

```
# target_metadata = Base.metadata
```

```
# Create migration
```

```
# alembic revision --autogenerate -m "create users table"
```

```
# Generated migration file (alembic/versions/xxxx_create_users.py):
```

```
def upgrade():
```

```
    op.create_table(
```

```
        'users',
```

```
        sa.Column('id', sa.Integer(), primary_key=True),
```

```
        sa.Column('name', sa.String(50), nullable=False),
```

```
        sa.Column('email', sa.String(100), unique=True),
```

```
    )
```

```
def downgrade():
```

```
    op.drop_table('users')
```



```
# Apply migration
# alembic upgrade head

# Rollback
# alembic downgrade -1

# Check current version
# alembic current
```

49. How do you perform Database Migrations?

Answer: Use Alembic with SQLAlchemy to create, apply, and rollback database schema changes.

Example:

```
# Step 1: Setup
# pip install alembic
# alembic init migrations

# Step 2: Configure migrations/env.py
# from app.database import Base
# target_metadata = Base.metadata

# Step 3: Create migration after model changes
# alembic revision --autogenerate -m "add phone column to users"

# Generated file:
from alembic import op
import sqlalchemy as sa
```

```

def upgrade():
    op.add_column('users', sa.Column('phone', sa.String(15), nullable=True))

def downgrade():
    op.drop_column('users', 'phone')

# Step 4: Apply
# alembic upgrade head      (apply all)
# alembic upgrade +1       (apply next one)

# Step 5: Rollback
# alembic downgrade -1     (rollback one)
# alembic downgrade base   (rollback all)

# History
# alembic history
# alembic current

```

50. What is CRUD in FastAPI?

Answer: CRUD = Create, Read, Update, Delete. These are the four basic operations for database records, mapped to HTTP methods: POST, GET, PUT/PATCH, DELETE.

Example:

```

from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.orm import Session
from pydantic import BaseModel
from typing import List

```

```
app = FastAPI()
```

```
# Schema
```

```
class TaskCreate(BaseModel):
```

```
    title: str
```

```
    completed: bool = False
```

```
class TaskResponse(TaskCreate):
```

```
    id: int
```

```
    class Config:
```

```
        from_attributes = True
```

```
# CRUD operations
```

```
# CREATE
```

```
@app.post("/tasks/", response_model=TaskResponse)
```

```
def create_task(task: TaskCreate, db: Session = Depends(get_db)):
```

```
    db_task = Task(**task.dict())
```

```
    db.add(db_task)
```

```
    db.commit()
```

```
    db.refresh(db_task)
```

```
    return db_task
```

```
# READ (all)
```

```
@app.get("/tasks/", response_model=List[TaskResponse])
```

```
def get_tasks(skip: int = 0, limit: int = 10, db: Session = Depends(get_db)):
```

```
    return db.query(Task).offset(skip).limit(limit).all()
```

```
# READ (one)
```

```
@app.get("/tasks/{task_id}", response_model=TaskResponse)
```

```

def get_task(task_id: int, db: Session = Depends(get_db)):
    task = db.query(Task).filter(Task.id == task_id).first()
    if not task:
        raise HTTPException(404, "Task not found")
    return task

# UPDATE
@app.put("/tasks/{task_id}", response_model=TaskResponse)
def update_task(task_id: int, task: TaskCreate, db: Session = Depends(get_db)):
    db_task = db.query(Task).filter(Task.id == task_id).first()
    if not db_task:
        raise HTTPException(404, "Task not found")
    for key, value in task.dict().items():
        setattr(db_task, key, value)
    db.commit()
    db.refresh(db_task)
    return db_task

# DELETE
@app.delete("/tasks/{task_id}")
def delete_task(task_id: int, db: Session = Depends(get_db)):
    task = db.query(Task).filter(Task.id == task_id).first()
    if not task:
        raise HTTPException(404, "Task not found")
    db.delete(task)
    db.commit()
    return {"message": "Task deleted"}

```

51. How do you implement Pagination?

Answer: Use skip and limit query parameters to control which records are returned.
Can also implement cursor-based pagination for large datasets.

Example:

```
from fastapi import FastAPI, Depends, Query
from pydantic import BaseModel
from typing import List
```

```
app = FastAPI()
```

```
# Offset-based pagination
```

```
@app.get("/items/")
```

```
def get_items(
```

```
    skip: int = Query(default=0, ge=0),
```

```
    limit: int = Query(default=10, ge=1, le=100)
```

```
):
```

```
    # db.query(Item).offset(skip).limit(limit).all()
```

```
    items = list(range(100)) # simulated data
```

```
    return {
```

```
        "items": items[skip: skip + limit],
```

```
        "total": len(items),
```

```
        "skip": skip,
```

```
        "limit": limit,
```

```
        "has_next": skip + limit < len(items)
```

```
    }
```

```
# Page-based pagination
```

```
@app.get("/users/")
```

```
def get_users(page: int = Query(default=1, ge=1), size: int = Query(default=10,
ge=1, le=50)):
```

```
    skip = (page - 1) * size
```

```

# total = db.query(User).count()
# users = db.query(User).offset(skip).limit(size).all()
total = 95 # simulated
return {
    "page": page,
    "size": size,
    "total": total,
    "total_pages": (total + size - 1) // size,
    "data": []
}

```

52. How do you implement Filtering and Sorting?

Answer: Use query parameters for filter criteria and sort fields. Apply them to database queries dynamically.

Example:

```

from fastapi import FastAPI, Query
from typing import Optional
from enum import Enum

```

```

app = FastAPI()

```

```

class SortOrder(str, Enum):
    asc = "asc"
    desc = "desc"

```

```

@app.get("/products/")
def get_products(
    # Filtering

```

```

category: Optional[str] = None,
min_price: Optional[float] = Query(None, ge=0),
max_price: Optional[float] = Query(None, ge=0),
in_stock: Optional[bool] = None,
# Sorting
sort_by: Optional[str] = Query(None, regex="^(name|price|created_at)$"),
order: SortOrder = SortOrder.asc,
# Pagination
skip: int = 0,
limit: int = 10
):
    # Build query dynamically
    # query = db.query(Product)
    # if category:
    #     query = query.filter(Product.category == category)
    # if min_price:
    #     query = query.filter(Product.price >= min_price)
    # if max_price:
    #     query = query.filter(Product.price <= max_price)
    # if in_stock is not None:
    #     query = query.filter(Product.in_stock == in_stock)
    # if sort_by:
    #     col = getattr(Product, sort_by)
    #     query = query.order_by(col.desc() if order == "desc" else col.asc())
    # return query.offset(skip).limit(limit).all()

    return {
        "filters": {"category": category, "min_price": min_price, "max_price":
max_price},
        "sort": {"by": sort_by, "order": order},
        "pagination": {"skip": skip, "limit": limit}

```

```
}
```

53. What is API Versioning?

Answer: API versioning allows you to maintain multiple versions of your API simultaneously. This lets you make breaking changes without affecting existing clients. Common approaches: URL path, header, or query parameter.

Example:

```
from fastapi import FastAPI, APIRouter, Header
```

```
app = FastAPI()
```

```
# Method 1: URL path versioning (most common)
```

```
v1_router = APIRouter(prefix="/api/v1")
```

```
v2_router = APIRouter(prefix="/api/v2")
```

```
@v1_router.get("/users")
```

```
def get_users_v1():
```

```
    return {"version": "v1", "users": [{"name": "Krishna"}]}
```

```
@v2_router.get("/users")
```

```
def get_users_v2():
```

```
    return {"version": "v2", "users": [{"name": "Krishna", "email": "k@test.com"}]}
```

```
app.include_router(v1_router)
```

```
app.include_router(v2_router)
```

```
# Method 2: Header-based versioning
```

```
@app.get("/users")
```



```
def get_users(api_version: str = Header(default="v1")):
    if api_version == "v2":
        return {"users": [{"name": "Krishna", "email": "k@test.com"}]}
    return {"users": [{"name": "Krishna"}]}
```

54. How do you structure a FastAPI project?

Answer: Organize code into modules: routes, models, schemas, services, and database configuration. Use APIRouter for route organization.

Example:

```
# Project structure:
# app/
#   __init__.py
#   main.py
#   config.py
#   database.py
#   models/
#     __init__.py
#     user.py
#   schemas/
#     __init__.py
#     user.py
#   routers/
#     __init__.py
#     users.py
#     items.py
#   services/
#     __init__.py
#     user_service.py
```

```
# dependencies.py
```

```
# --- app/main.py ---
```

```
from fastapi import FastAPI
```

```
from app.routers import users, items
```

```
from app.database import engine, Base
```

```
app = FastAPI(title="My App")
```

```
Base.metadata.create_all(bind=engine)
```

```
app.include_router(users.router)
```

```
app.include_router(items.router)
```

```
# --- app/routers/users.py ---
```

```
from fastapi import APIRouter, Depends
```

```
from app.dependencies import get_db
```

```
router = APIRouter(prefix="/users", tags=["users"])
```

```
@router.get("/")
```

```
def get_users(db=Depends(get_db)):
```

```
    return []
```

```
@router.get("/{user_id}")
```

```
def get_user(user_id: int, db=Depends(get_db)):
```

```
    return {"user_id": user_id}
```

```
# --- app/config.py ---
```

```
from pydantic_settings import BaseSettings
```

```

class Settings(BaseSettings):
    database_url: str = "sqlite:///./app.db"
    secret_key: str = "secret"
    class Config:
        env_file = ".env"

settings = Settings()

```

55. What is Dependency Override?

Answer: Dependency override replaces a dependency with a different implementation, primarily used in testing. It lets you swap real databases/services with mocks.

Example:

```

from fastapi import FastAPI, Depends
from fastapi.testclient import TestClient

```

```

app = FastAPI()

```

```

# Real dependency

```

```

def get_db():
    return "real_database"

```

```

@app.get("/items/")

```

```

def get_items(db=Depends(get_db)):
    return {"db": db}

```

```

# Override in tests

```

```

def override_get_db():

```

```

    return "test_database"

app.dependency_overrides[get_db] = override_get_db

client = TestClient(app)
response = client.get("/items/")
print(response.json()) # {"db": "test_database"}

# Reset overrides
app.dependency_overrides = {}

# Practical test example
# def test_get_items():
#     app.dependency_overrides[get_db] = override_get_db
#     response = client.get("/items/")
#     assert response.status_code == 200
#     app.dependency_overrides = {}

```

56. How do you test FastAPI applications?

Answer: Use TestClient from fastapi.testclient (based on httpx) with pytest. Create test functions that make HTTP requests to your app.

Example:

```

from fastapi import FastAPI

from fastapi.testclient import TestClient

from pydantic import BaseModel

app = FastAPI()

```

```
class Item(BaseModel):
    name: str
    price: float

items_db = []

@app.post("/items/", status_code=201)
def create_item(item: Item):
    items_db.append(item.dict())
    return item

@app.get("/items/")
def get_items():
    return items_db

# --- Tests ---
client = TestClient(app)

def test_create_item():
    response = client.post("/items/", json={"name": "Phone", "price": 999.0})
    assert response.status_code == 201
    assert response.json()["name"] == "Phone"

def test_get_items():
    response = client.get("/items/")
    assert response.status_code == 200
    assert isinstance(response.json(), list)

def test_invalid_item():
    response = client.post("/items/", json={"name": "Phone"}) # missing price
```

```
assert response.status_code == 422 # validation error
```

```
# Run: pytest test_main.py -v
```

57. What is TestClient?

Answer: TestClient is FastAPI's testing utility (based on httpx/requests). It makes HTTP requests to your app without starting a real server, making tests fast and isolated.

Example:

```
from fastapi import FastAPI, HTTPException
from fastapi.testclient import TestClient

app = FastAPI()

@app.get("/users/{user_id}")
def get_user(user_id: int):
    if user_id == 0:
        raise HTTPException(404, "User not found")
    return {"user_id": user_id, "name": "Krishna"}

client = TestClient(app)

# Test successful response
def test_get_user_success():
    response = client.get("/users/1")
    assert response.status_code == 200
    data = response.json()
    assert data["user_id"] == 1
```

```

    assert data["name"] == "Krishna"

# Test error response
def test_get_user_not_found():
    response = client.get("/users/0")
    assert response.status_code == 404

# Test with headers
def test_with_auth():
    response = client.get("/protected", headers={"Authorization": "Bearer token123"})
    assert response.status_code == 200

# Test with query params
def test_with_params():
    response = client.get("/items/", params={"skip": 0, "limit": 5})
    assert response.status_code == 200

```

58. What is PyTest with FastAPI?

Answer: PyTest is the recommended testing framework for FastAPI. Use fixtures for setup/teardown, parametrize for multiple test cases, and conftest.py for shared configurations.

Example:

```

import pytest

from fastapi.testclient import TestClient
from main import app

# Fixture for test client

```

```

@pytest.fixture
def client():
    return TestClient(app)

# Fixture for test database
@pytest.fixture
def test_db():
    # Setup test database
    Base.metadata.create_all(bind=test_engine)
    yield TestSessionLocal()
    # Teardown
    Base.metadata.drop_all(bind=test_engine)

# Tests using fixtures
def test_root(client):
    response = client.get("/")
    assert response.status_code == 200

def test_create_user(client):
    response = client.post("/users/", json={"name": "Krishna", "email":
"k@test.com"})
    assert response.status_code == 201
    assert response.json()["name"] == "Krishna"

# Parametrized tests
@pytest.mark.parametrize("user_id,status", [(1, 200), (999, 404), (0, 404)])
def test_get_user(client, user_id, status):
    response = client.get(f"/users/{user_id}")
    assert response.status_code == status

# Run: pytest tests/ -v --cov=app

```


59. How do you mock dependencies in tests?

Answer: Use `app.dependency_overrides` to replace real dependencies with mock/test versions during testing.

Example:

```
import pytest

from fastapi import FastAPI, Depends
from fastapi.testclient import TestClient

app = FastAPI()

# Real dependencies
def get_db():
    return "production_db"

def get_current_user():
    return {"id": 1, "name": "Real User"}

@app.get("/profile")
def get_profile(user=Depends(get_current_user), db=Depends(get_db)):
    return {"user": user, "db": db}

# Test with mocked dependencies
@pytest.fixture
def client():
    # Mock dependencies
    def mock_db():
        return "test_db"
```

```

def mock_user():
    return {"id": 99, "name": "Test User"}

app.dependency_overrides[get_db] = mock_db
app.dependency_overrides[get_current_user] = mock_user

yield TestClient(app)

# Cleanup
app.dependency_overrides = {}

def test_profile(client):
    response = client.get("/profile")
    assert response.status_code == 200
    data = response.json()
    assert data["user"]["name"] == "Test User"
    assert data["db"] == "test_db"

```

60. How do you deploy FastAPI applications?

Answer: Deploy using Docker + cloud platforms (AWS, GCP, Azure), or directly with Uvicorn/Gunicorn behind Nginx.

Example:

```

# Method 1: Direct deployment with Gunicorn + Uvicorn workers
# pip install gunicorn uvicorn
# gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker -b 0.0.0.0:8000

# Method 2: Docker

```

```
# --- Dockerfile ---  
# FROM python:3.11-slim  
# WORKDIR /app  
# COPY requirements.txt .  
# RUN pip install --no-cache-dir -r requirements.txt  
# COPY . .  
# CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Method 3: Docker Compose

```
# --- docker-compose.yml ---  
# version: '3.8'  
# services:  
#   api:  
#     build: .  
#     ports:  
#       - "8000:8000"  
#     environment:  
#       - DATABASE_URL=postgresql://user:pass@db/mydb  
#   db:  
#     image: postgres:15  
#     environment:  
#       - POSTGRES_PASSWORD=pass
```

Method 4: AWS (ECS/Fargate, Lambda, EC2)

Method 5: Render, Railway, or Heroku (simple PaaS)

Production checklist:

- Use environment variables for secrets

- Enable HTTPS

- Set up logging

```
# - Use health check endpoint
# - Configure CORS properly
# - Set up monitoring
```

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/health")
```

```
def health_check():
```

```
    return {"status": "healthy"}
```

61. FastAPI with Docker?

Answer: Containerize FastAPI using Docker for consistent deployments across environments.

Example:

```
# --- Dockerfile ---
```

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
EXPOSE 8000
```

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# --- requirements.txt ---
```

```
# fastapi==0.104.0
```

```
# uvicorn==0.24.0
```

```
# sqlalchemy==2.0.0
```

```
# pydantic==2.5.0
```

```
# --- docker-compose.yml ---
```

```
# version: '3.8'
```

```
# services:
```

```
#   web:
```

```
#     build: .
```

```
#     ports:
```

```
#       - "8000:8000"
```

```
#     depends_on:
```

```
#       - db
```

```
#     environment:
```

```
#       - DATABASE_URL=postgresql://user:pass@db:5432/appdb
```

```
#   db:
```

```
#     image: postgres:15
```

```
#     environment:
```

```
#       POSTGRES_USER: user
```

```
#       POSTGRES_PASSWORD: pass
```

```
#       POSTGRES_DB: appdb
```

```
#     volumes:
```

```
#       - pgdata:/var/lib/postgresql/data
```

```
# volumes:
```

```
#   pgdata:
```

```
# Build and run:
```

```
# docker build -t fastapi-app .  
# docker run -p 8000:8000 fastapi-app  
# OR  
# docker-compose up --build
```

62. FastAPI with Nginx?

Answer: Nginx acts as a reverse proxy in front of FastAPI, handling SSL, load balancing, and static files.

Example:

```
# --- nginx.conf ---  
# upstream fastapi {  
#     server web:8000;  
# }  
#  
# server {  
#     listen 80;  
#     server_name myapi.com;  
#  
#     location / {  
#         proxy_pass http://fastapi;  
#         proxy_set_header Host $host;  
#         proxy_set_header X-Real-IP $remote_addr;  
#         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
#         proxy_set_header X-Forwarded-Proto $scheme;  
#     }  
#  
#     location /static {  
#         alias /app/static;
```

```
# }
```

```
# }
```

```
# --- docker-compose.yml with Nginx ---
```

```
# version: '3.8'
```

```
# services:
```

```
# web:
```

```
# build: .
```

```
# expose:
```

```
# - "8000"
```

```
# nginx:
```

```
# image: nginx:latest
```

```
# ports:
```

```
# - "80:80"
```

```
# - "443:443"
```

```
# volumes:
```

```
# - ./nginx.conf:/etc/nginx/conf.d/default.conf
```

```
# depends_on:
```

```
# - web
```

```
# SSL with Nginx (HTTPS):
```

```
# server {
```

```
# listen 443 ssl;
```

```
# ssl_certificate /etc/ssl/cert.pem;
```

```
# ssl_certificate_key /etc/ssl/key.pem;
```

```
# location / {
```

```
#     proxy_pass http://fastapi;
```

```
# }
```

```
# }
```

63. How do you deploy FastAPI on AWS?

Answer: Multiple AWS options: ECS/Fargate (containers), Lambda (serverless), EC2 (VMs), or Elastic Beanstalk.

Example:

Option 1: AWS ECS with Fargate (recommended for containers)

- Push Docker image to ECR

- Create ECS Task Definition

- Create ECS Service with Fargate launch type

- Put ALB in front

Option 2: AWS Lambda with Mangum

pip install mangum

from fastapi import FastAPI

from mangum import Mangum

app = FastAPI()

@app.get("/")

def root():

return {"message": "Running on Lambda!"}

Lambda handler

handler = Mangum(app)

Option 3: EC2

- Launch EC2 instance

- Install Python, pip, dependencies

- Run with Gunicorn + Uvicorn workers

- Use systemd for process management

- Put behind ALB or Nginx

--- systemd service file (/etc/systemd/system/fastapi.service) ---

[Unit]

Description=FastAPI App

After=network.target

#

[Service]

User=ubuntu

WorkingDirectory=/home/ubuntu/app

ExecStart=/home/ubuntu/app/venv/bin/gunicorn main:app -w 4 -k
uvicorn.workers.UvicornWorker -b 0.0.0.0:8000

Restart=always

#

[Install]

WantedBy=multi-user.target

Commands:

sudo systemctl start fastapi

sudo systemctl enable fastapi

64. How do you optimize FastAPI performance?

Answer: Use async routes, connection pooling, caching, background tasks, proper indexing, and multiple workers.

Example:

from fastapi import FastAPI

from functools import lru_cache

```
import asyncio
```

```
app = FastAPI()
```

```
# 1. Use async routes for I/O operations
```

```
@app.get("/async-data")
```

```
async def get_data():
```

```
    await asyncio.sleep(0) # non-blocking
```

```
    return {"data": "fast"}
```

```
# 2. Caching with lru_cache
```

```
@lru_cache(maxsize=100)
```

```
def get_settings():
```

```
    return {"db_url": "postgresql://...", "secret": "key"}
```

```
# 3. Response caching
```

```
from fastapi.responses import JSONResponse
```

```
cache = {}
```

```
@app.get("/cached/{item_id}")
```

```
async def get_cached_item(item_id: int):
```

```
    if item_id in cache:
```

```
        return cache[item_id]
```

```
    result = {"item_id": item_id, "data": "expensive computation"}
```

```
    cache[item_id] = result
```

```
    return result
```

```
# 4. Connection pooling (SQLAlchemy)
```

```
# engine = create_engine(url, pool_size=20, max_overflow=10)
```

5. Run with multiple workers

```
# gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker
```

6. Use ORJSONResponse for faster JSON serialization

```
from fastapi.responses import ORJSONResponse
```

```
@app.get("/fast", response_class=ORJSONResponse)
```

```
async def fast_endpoint():
```

```
    return {"data": "serialized with orjson"}
```

7. Pagination to limit data

8. Proper database indexes

9. Gzip middleware for compression

65. What are Async Routes?

Answer: Async routes use `async/await` to handle I/O-bound operations without blocking the event loop. They allow concurrent request handling, improving throughput for operations like database calls, HTTP requests, and file I/O.

Example:

```
from fastapi import FastAPI
```

```
import asyncio
```

```
import httpx
```

```
app = FastAPI()
```

```
# Async route - non-blocking
```

```
@app.get("/async")
```

```

async def async_route():
    await asyncio.sleep(1) # non-blocking sleep
    return {"message": "async response"}

# Sync route - runs in threadpool
@app.get("/sync")
def sync_route():
    import time
    time.sleep(1) # blocking - FastAPI runs in thread
    return {"message": "sync response"}

# Async HTTP call
@app.get("/fetch")
async def fetch_external():
    async with httpx.AsyncClient() as client:
        response = await client.get("https://api.example.com/data")
        return response.json()

# Multiple async operations concurrently
@app.get("/parallel")
async def parallel_calls():
    async with httpx.AsyncClient() as client:
        tasks = [
            client.get("https://api.example.com/users"),
            client.get("https://api.example.com/posts"),
            client.get("https://api.example.com/comments"),
        ]
        responses = await asyncio.gather(*tasks)
        return {"results": [r.status_code for r in responses]}

```

```
# Rule: Use async def when you have await calls
# Use def (sync) for CPU-bound or non-async libraries
```

66. What is Event Loop?

Answer: The event loop is the core of async programming. It schedules and runs coroutines, handles I/O events, and manages callbacks. Python's asyncio provides the event loop. FastAPI/Uvicorn runs the event loop that processes all async requests.

Example:

```
import asyncio

# How event loop works conceptually
async def task1():
    print("Task 1 start")
    await asyncio.sleep(2) # gives control back to event loop
    print("Task 1 end")

async def task2():
    print("Task 2 start")
    await asyncio.sleep(1)
    print("Task 2 end")

async def main():
    # Event loop runs both tasks concurrently
    await asyncio.gather(task1(), task2())
    # Output:
    # Task 1 start
    # Task 2 start
```

```

# Task 2 end (after 1s)
# Task 1 end (after 2s)
# Total: ~2 seconds, not 3

asyncio.run(main())

# In FastAPI, Uvicorn manages the event loop
# Each request is a coroutine in the event loop
# When one request awaits I/O, others can be processed

from fastapi import FastAPI
app = FastAPI()

@app.get("/")
async def root():
    # This runs in the event loop
    await asyncio.sleep(0) # yield to other requests
    return {"message": "processed"}

```

67. How do you use Async Database Operations?

Answer: Use async database libraries like asyncpg, databases, or SQLAlchemy with async engine for non-blocking database operations.

Example:

```

from fastapi import FastAPI, Depends
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker, declarative_base
from sqlalchemy import Column, Integer, String, select

```

```

# Async engine
DATABASE_URL = "postgresql+asyncpg://user:pass@localhost/db"
async_engine = create_async_engine(DATABASE_URL, pool_size=20)
AsyncSessionLocal = sessionmaker(async_engine, class_=AsyncSession,
expire_on_commit=False)

Base = declarative_base()

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    name = Column(String)

# Async dependency
async def get_db():
    async with AsyncSessionLocal() as session:
        yield session

app = FastAPI()

# Async CRUD operations
@app.get("/users/")
async def get_users(db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(User))
    return result.scalars().all()

@app.post("/users/")
async def create_user(name: str, db: AsyncSession = Depends(get_db)):
    user = User(name=name)
    db.add(user)
    await db.commit()

```

```

    await db.refresh(user)

    return {"id": user.id, "name": user.name}

# Using 'databases' library (alternative)
# from databases import Database

# database = Database(DATABASE_URL)

# await database.execute(query="INSERT INTO users(name) VALUES(:name)",
values={"name": "Krishna"})

```

68. What are Lifespan Events?

Answer: Lifespan events run code at application startup and shutdown. They're used to initialize/cleanup resources like database connections, caches, ML models. Use the lifespan context manager (recommended over deprecated `on_event`).

Example:

```

from fastapi import FastAPI

from contextlib import asynccontextmanager

# Shared resources
resources = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup: runs before app starts accepting requests
    print("Starting up...")
    resources["db"] = "database_connection"
    resources["cache"] = {}
    # Load ML model, connect to DB, etc.
    yield

```



```

# Shutdown: runs when app is stopping
print("Shutting down...")
resources.clear()
# Close connections, cleanup resources

app = FastAPI(lifespan=lifespan)

@app.get("/")
def root():
    return {"db": resources.get("db"), "status": "running"}

```

69. What are Startup and Shutdown Events?

Answer: Startup events run when the application starts (before accepting requests). Shutdown events run when the application stops. Note: `on_event` is deprecated; use `lifespan` instead.

Example:

```

from fastapi import FastAPI

app = FastAPI()

# Deprecated but still works
@app.on_event("startup")
async def startup_event():
    print("App is starting...")
    # Initialize database connection pool
    # Load configuration
    # Warm up caches

```

```
@app.on_event("shutdown")
async def shutdown_event():
    print("App is shutting down...")
    # Close database connections
    # Flush logs
    # Cleanup resources

# Modern way: lifespan (recommended)
from contextlib import asynccontextmanager
```

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup
    print("Startup: connecting to DB")
    yield
    # Shutdown
    print("Shutdown: closing connections")
```

```
app = FastAPI(lifespan=lifespan)
```

```
@app.get("/")
def root():
    return {"status": "running"}
```

70. What are Best Practices for FastAPI Development?

Answer:

1. Project structure - organize into routers, models, schemas, services
2. Use Pydantic for all data validation

3. Use dependency injection for shared logic
4. Use async for I/O-bound operations
5. Proper error handling with custom exceptions
6. Environment variables for configuration
7. Write tests with pytest + TestClient
8. Use type hints everywhere
9. Document API with descriptions and examples
10. Secure endpoints with authentication

Example:

```
from fastapi import FastAPI, Depends, HTTPException
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings
from typing import Optional
from contextlib import asynccontextmanager
```

1. Configuration from environment

```
class Settings(BaseSettings):
```

```
    database_url: str
    secret_key: str
    debug: bool = False
    class Config:
        env_file = ".env"
```

```
settings = Settings()
```

2. Lifespan for startup/shutdown

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    print("Starting up")
```

```
yield
print("Shutting down")
```

3. App with metadata

```
app = FastAPI(
    title="Production API",
    version="1.0.0",
    lifespan=lifespan
)
```

4. Pydantic schemas with validation

```
class UserCreate(BaseModel):
    name: str = Field(min_length=2, max_length=50)
    email: str = Field(pattern=r"^\[\w.\]+\@\[\w\]+\.\[\w\]+$")
```

```
class UserResponse(BaseModel):
    id: int
    name: str
    email: str
```

5. Dependency for common logic

```
def get_db():
    db = "session"
    try:
        yield db
    finally:
        pass # close session
```

6. Service layer (business logic separate from routes)

```
class UserService:
```

```
def __init__(self, db):
```

```
    self.db = db
```

```
def create_user(self, user: UserCreate):
```

```
    return {"id": 1, **user.dict()}}
```

7. Route with proper error handling

```
@app.post("/users/", response_model=UserResponse, status_code=201)
```

```
def create_user(user: UserCreate, db=Depends(get_db)):
```

```
    service = UserService(db)
```

```
    try:
```

```
        return service.create_user(user)
```

```
    except Exception as e:
```

```
        raise HTTPException(500, detail=str(e))
```

8. Health check endpoint

```
@app.get("/health")
```

```
def health():
```

```
    return {"status": "healthy", "version": "1.0.0"}
```

```
=====
```

ADDITIONAL TOPICS - FROM OTHER FILES (MISSING IN ABOVE)

```
=====
```

81. What is Singleton Design Pattern?

```
-----
```

Answer: Singleton ensures a class has only one instance throughout the program. Useful for database connections, configuration managers, logging.

Example:

```
class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if not cls._instance:
            cls._instance = super().__new__(cls, *args, **kwargs)
        return cls._instance

s1 = Singleton()
s2 = Singleton()
print(s1 is s2) # True (same instance)
```

82. What are Metaclasses?

Answer: Metaclasses are "classes of classes" - they define how classes behave. type is the default metaclass. They're used for framework-level development to customize class creation.

Example:

```
class Meta(type):
    def __new__(cls, name, bases, dct):
        dct['created_by'] = 'Meta'
        return super().__new__(cls, name, bases, dct)

class MyClass(metaclass=Meta):
    pass
```

```
print(MyClass.created_by) # 'Meta'
```

83. What is SOLID Principle?

Answer:

- S: Single Responsibility - class should have one job
- O: Open/Closed - open for extension, closed for modification
- L: Liskov Substitution - subclasses replaceable for parent
- I: Interface Segregation - many small interfaces > one big
- D: Dependency Inversion - depend on abstractions, not concretions

Example:

```
from abc import ABC, abstractmethod
```

```
# Single Responsibility
```

```
class UserRepository:
```

```
    def save(self, user): pass
```

```
class EmailService:
```

```
    def send(self, email, msg): pass
```

```
# Open/Closed - extend via new classes, don't modify existing
```

```
class Shape(ABC):
```

```
    @abstractmethod
```

```
    def area(self): pass
```

```
class Circle(Shape):
```

```
    def __init__(self, r): self.r = r
```

```
def area(self): return 3.14 * self.r ** 2
```

Dependency Inversion

```
class NotificationService:
```

```
    def __init__(self, sender: ABC): # depends on abstraction
        self.sender = sender
```

84. Mutable Default Arguments Trap

Answer: Default arguments are evaluated once at function definition, not each call. Using mutable defaults (list, dict) causes shared state across calls.

Example:

BUG - shared list across calls

```
def add_item_bad(item, box=[]):
    box.append(item)
    return box
```

```
print(add_item_bad(1)) # [1]
```

```
print(add_item_bad(2)) # [1, 2] - Bug! Expected [2]
```

FIX - use None as default

```
def add_item_good(item, box=None):
    if box is None:
        box = []
    box.append(item)
    return box
```

```
print(add_item_good(1)) # [1]
```

```
print(add_item_good(2)) # [2] - Correct!
```


85. LEGB Rule (Variable Scoping)

Answer: Python resolves variables in order: Local → Enclosing → Global → Built-in.
Use nonlocal for enclosing scope, global for module scope.

Example:

```
x = "global"
```

```
def outer():
```

```
    x = "enclosing"
```

```
    def inner():
```

```
        # x = "local" # if uncommented, this is used
```

```
        print(x) # "enclosing" (LEGB: no local, checks enclosing)
```

```
    inner()
```

```
outer()
```

```
# nonlocal vs global
```

```
def counter():
```

```
    count = 0
```

```
    def increment():
```

```
        nonlocal count # modifies enclosing scope
```

```
        count += 1
```

```
        return count
```

```
    return increment
```

```
c = counter()
```

```
print(c()) # 1
```

```
print(c()) # 2
```

86. Dictionary Internals & Hashing

Answer: Python dicts use hash tables with open addressing. Average lookup is $O(1)$. Keys must be hashable (immutable + `__hash__`). Since Python 3.7, dicts are insertion-ordered.

Example:

```
# Only hashable types can be keys
d = {(1, 2): "tuple key"}    # OK - tuple is hashable
# d = {[1, 2]: "list key"}   # TypeError - list not hashable
```

```
# Custom hashable class
```

```
class Point:
```

```
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __hash__(self):
        return hash((self.x, self.y))
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y
```

```
p = Point(1, 2)
```

```
d = {p: "origin"}
```

87. List Comprehension vs Generator Expression (Memory)

Answer: List comprehension (`[]`) is eager - loads all into memory. Generator expression (`()`) is lazy - produces one at a time. Use generators for large datasets.

Example:

```
import sys
```

```
# List - all in memory
```

```
lst = [x**2 for x in range(1000000)]
```

```
print(sys.getsizeof(lst)) # ~8 MB
```

```
# Generator - negligible memory
```

```
gen = (x**2 for x in range(1000000))
```

```
print(sys.getsizeof(gen)) # ~120 bytes
```

88. collections.deque for $O(1)$ insert at front

Answer: `list.insert(0, x)` is $O(n)$. Use `collections.deque` for $O(1)$ `appendleft/popleft`.

Example:

```
from collections import deque
```

```
# Bad -  $O(n)$  for each insert at 0
```

```
lst = []
```

```
for i in range(10000):
```

```
    lst.insert(0, i) # shifts all elements
```

```
# Good -  $O(1)$  for appendleft
```

```
dq = deque()
```

```
for i in range(10000):
```

```
    dq.appendleft(i) # constant time
```

89. Composition vs Inheritance

Answer: Inheritance = "is-a" (Dog is Animal). Composition = "has-a" (Car has Engine). Composition is preferred to avoid deep hierarchies and enable runtime flexibility.

Example:

```
# Inheritance (tight coupling)

class Animal:
    def speak(self): pass

class Dog(Animal):
    def speak(self): return "Woof"

# Composition (flexible, testable)

class Engine:
    def start(self): return "Engine started"

class Car:
    def __init__(self, engine: Engine):
        self.engine = engine # has-a relationship
    def start(self):
        return self.engine.start()

car = Car(Engine())
print(car.start()) # Engine started
```

90. How to throw exception from calling function

Answer: Use raise to throw exceptions. Exceptions propagate up the call stack until caught.

Example:

```
def validate_age(age):  
    if age < 0:  
        raise ValueError(f"Age cannot be negative: {age}")  
    return age  
  
def create_user(name, age):  
    validated_age = validate_age(age) # exception propagates up  
    return {"name": name, "age": validated_age}  
  
try:  
    user = create_user("Krishna", -5)  
except ValueError as e:  
    print(f"Caught: {e}") # Caught: Age cannot be negative: -5
```

91. Passing variable from child to parent class (Inheritance)

Answer: Use `super().__init__()` to pass variables from child to parent constructor.

Example:

```
class Parent:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
  
class Child(Parent):  
    def __init__(self, name, age, school):  
        super().__init__(name, age) # pass to parent
```

```
self.school = school
```

```
c = Child("Krishna", 10, "ABC School")
```

```
print(c.name, c.age, c.school) # Krishna 10 ABC School
```

92. Python Profiling Tools

Answer: Use profiling tools to find performance bottlenecks:

- cProfile: overall function-level profiling
- line_profiler: line-by-line analysis
- memory_profiler: memory usage
- py-spy/scalene: production-safe sampling

Example:

```
import cProfile
```

```
def slow_function():
```

```
    total = 0
```

```
    for i in range(1000000):
```

```
        total += i ** 2
```

```
    return total
```

```
# Profile the function
```

```
cProfile.run('slow_function()')
```

```
# Or save to file
```

```
# cProfile.run('slow_function()', 'output.prof')
```

```
# memory profiler (install: pip install memory-profiler)
```

```
# @profile
# def memory_heavy():
#     big_list = [i for i in range(10000000)]
#     return sum(big_list)
```

```
=====
=====
CODING PROBLEMS (FROM python-programing-questions 1.py)
=====
=====
```

93. Reverse Words in a Sentence

Example:

```
def reverse_words(sentence):
    return " ".join(sentence.split()[::-1])

print(reverse_words("I love Python programming"))
# Output: "programming Python love I"
```

94. Find the Missing Number in a List

Example:

```
def find_missing(numbers, n):
    expected_sum = n * (n + 1) // 2
    return expected_sum - sum(numbers)

print(find_missing([1, 2, 4, 5], 5)) # 3
```

95. Count Character Frequency Without Collections

Example:

```
def char_frequency(s):
    freq = {}
    for char in s:
        freq[char] = freq.get(char, 0) + 1
    return freq

print(char_frequency("banana")) # {'b': 1, 'a': 3, 'n': 2}
```

96. LRU Cache Implementation

Example:

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.cache = OrderedDict()
        self.capacity = capacity

    def get(self, key):
        if key not in self.cache:
            return -1
        self.cache.move_to_end(key)
        return self.cache[key]
```



```

def put(self, key, value):
    if key in self.cache:
        self.cache.move_to_end(key)
    self.cache[key] = value
    if len(self.cache) > self.capacity:
        self.cache.popitem(last=False)

lru = LRUCache(2)
lru.put(1, 100)
lru.put(2, 200)
print(lru.get(1)) # 100
lru.put(3, 300) # evicts key 2
print(lru.get(2)) # -1

```

97. Detect Cycle in a Linked List (Floyd's Algorithm)

Example:

```

class ListNode:
    def __init__(self, val=0):
        self.val = val
        self.next = None

def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow == fast:
        return True

```

```
return False
```

98. Find All Unique Subsets

Example:

```
def subsets(nums):
    result = [[]]
    nums.sort()
    for num in nums:
        result += [subset + [num] for subset in result
                   if subset + [num] not in result]
    return result
```

```
print(subsets([1, 2, 2]))
# [[], [1], [2], [1, 2], [2, 2], [1, 2, 2]]
```

99. Simple Event Emitter

Example:

```
class EventEmitter:
    def __init__(self):
        self.listeners = {}

    def on(self, event, callback):
        self.listeners.setdefault(event, []).append(callback)

    def emit(self, event, *args):
        for cb in self.listeners.get(event, []):
```

```
cb(*args)
```

```
emitter = EventEmitter()
emitter.on("greet", lambda name: print(f"Hello {name}"))
emitter.emit("greet", "Alex") # Hello Alex
```

100. Valid Parentheses (LeetCode 20)

Example:

```
def is_valid(s):
    stack = []
    mapping = {'(': ')', ')': '(', '[': ']', ']': '['}
    for char in s:
        if char in mapping.values():
            stack.append(char)
        elif char in mapping:
            if not stack or stack[-1] != mapping[char]:
                return False
            stack.pop()
    return not stack
```

```
print(is_valid("()[]{}")) # True
print(is_valid("()"))      # False
```

101. Merge Two Sorted Lists

Example:

```
def merge_sorted(list1, list2):
```

```

i = j = 0
merged = []
while i < len(list1) and j < len(list2):
    if list1[i] < list2[j]:
        merged.append(list1[i])
        i += 1
    else:
        merged.append(list2[j])
        j += 1
merged.extend(list1[i:])
merged.extend(list2[j:])
return merged

```

```

print(merge_sorted([1, 2, 4], [1, 3, 4])) # [1, 1, 2, 3, 4, 4]

```

102. Multithreading: Sum of Squares

Example:

```

import threading

def sum_squares_chunk(nums, results, index):
    results[index] = sum(x**2 for x in nums)

numbers = [1, 2, 3, 4]
mid = len(numbers) // 2
results = [0, 0]

t1 = threading.Thread(target=sum_squares_chunk, args=(numbers[:mid], results,
0))

```

```

    t2 = threading.Thread(target=sum_squares_chunk, args=(numbers[mid:], results,
1))
    t1.start()
    t2.start()
    t1.join()
    t2.join()

print(f"Sum of squares: {sum(results)}") # 30

```

```

=====
=====
DJANGO INTERVIEW QUESTIONS WITH ANSWERS
=====
=====

```

103. How does Django work? Explain MVT.

Answer: Django follows MVT (Model-View-Template) architecture.

- Model: Handles database structure and ORM
- View: Contains business logic, fetches data, returns HTTP response
- Template: Handles frontend presentation (HTML)
- Request flow: Request → URL Dispatcher → View → Model → View → Template → Response

Example:

```

# --- models.py ---

from django.db import models

class Product(models.Model):
    name = models.CharField(max_length=100)

```

```

    price = models.DecimalField(max_digits=10, decimal_places=2)

# --- views.py ---
from django.shortcuts import render
from .models import Product

def product_list(request):
    products = Product.objects.all()
    return render(request, 'products/list.html', {'products': products})

# --- urls.py ---
from django.urls import path
from . import views

urlpatterns = [
    path('products/', views.product_list, name='product_list'),
]

# --- templates/products/list.html ---
# {% for product in products %}
#   <p>{{ product.name }} - ${{ product.price }}</p>
# {% endfor %}

```

104. Can we configure multiple databases in Django? How?

Answer: Yes. Add multiple DB entries in settings.py under DATABASES. Use database routers to control which model uses which database.

Example:

```

# --- settings.py ---

```

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql',  
        'NAME': 'primary_db',  
        'HOST': 'localhost',  
    },  
    'analytics': {  
        'ENGINE': 'django.db.backends.postgresql',  
        'NAME': 'analytics_db',  
        'HOST': 'localhost',  
    }  
}
```

```
# --- routers.py ---
```

```
class AnalyticsRouter:  
    def db_for_read(self, model, **hints):  
        if model._meta.app_label == 'analytics':  
            return 'analytics'  
        return 'default'  
  
    def db_for_write(self, model, **hints):  
        if model._meta.app_label == 'analytics':  
            return 'analytics'  
        return 'default'
```

```
# --- settings.py ---
```

```
DATABASE_ROUTERS = ['myapp.routers.AnalyticsRouter']
```

```
# Manual usage with using()
```

```
# Product.objects.using('analytics').all()
```

105. Difference between makemigrations and migrate?

Answer:

- makemigrations: Scans models, creates migration files (Python files describing schema changes)

- migrate: Reads migration files and applies them to the database (creates/alters tables)

Example:

Step 1: Create migration files after model changes

python manage.py makemigrations

Output: migrations/0001_initial.py

Step 2: Apply migrations to database

python manage.py migrate

Other useful commands:

python manage.py showmigrations # list all migrations

python manage.py migrate app_name 0001 # migrate to specific version

python manage.py migrate app_name zero # rollback all

106. What are Serializers in Django REST Framework?

Answer: Serializers convert complex data types (Django models, querysets) to/from JSON. They handle validation and transformation of data for API input/output.

Example:

```
from rest_framework import serializers
```



```
from .models import User
```

```
# Model Serializer
```

```
class UserSerializer(serializers.ModelSerializer):
```

```
    class Meta:
```

```
        model = User
```

```
        fields = ['id', 'username', 'email']
```

```
        read_only_fields = ['id']
```

```
# Usage in view
```

```
from rest_framework.views import APIView
```

```
from rest_framework.response import Response
```

```
class UserListView(APIView):
```

```
    def get(self, request):
```

```
        users = User.objects.all()
```

```
        serializer = UserSerializer(users, many=True)
```

```
        return Response(serializer.data)
```

```
    def post(self, request):
```

```
        serializer = UserSerializer(data=request.data)
```

```
        if serializer.is_valid():
```

```
            serializer.save()
```

```
            return Response(serializer.data, status=201)
```

```
        return Response(serializer.errors, status=400)
```

107. What are Middlewares in Django? How do they execute?

Answer: Middleware is a layer that processes requests/responses globally.
Execution order:

Request → Middleware 1 → Middleware 2 → View → Middleware 2 → Middleware 1
→ Response

Used for: authentication, security, logging, sessions, CORS.

Example:

```
# --- middleware.py ---
```

```
import time
```

```
class RequestTimingMiddleware:
```

```
    def __init__(self, get_response):
```

```
        self.get_response = get_response
```

```
    def __call__(self, request):
```

```
        # Before view (request phase)
```

```
        start = time.time()
```

```
        response = self.get_response(request) # calls next middleware/view
```

```
        # After view (response phase)
```

```
        duration = time.time() - start
```

```
        response['X-Request-Duration'] = str(duration)
```

```
        return response
```

```
# --- settings.py ---
```

```
MIDDLEWARE = [
```

```
    'django.middleware.security.SecurityMiddleware',
```

```
    'myapp.middleware.RequestTimingMiddleware', # custom
```

```
    'django.contrib.sessions.middleware.SessionMiddleware',
```

```
    'django.middleware.common.CommonMiddleware',
```

```
    'django.middleware.csrf.CsrfViewMiddleware',
```

```
'django.contrib.auth.middleware.AuthenticationMiddleware',  
]
```

108. How do you deploy Django? Which server?

Answer: Typical production setup:

- Gunicorn or uWSGI as WSGI application server
- Nginx as reverse proxy (handles static files, SSL, load balancing)
- Deploy on AWS EC2, ECS, Elastic Beanstalk, or Docker

Example:

```
# Install
```

```
# pip install gunicorn
```

```
# Run with Gunicorn
```

```
# gunicorn myproject.wsgi:application --bind 0.0.0.0:8000 --workers 4
```

```
# --- Nginx config ---
```

```
# server {
```

```
#     listen 80;
```

```
#     server_name myapp.com;
```

```
#
```

```
#     location /static/ {
```

```
#         alias /home/ubuntu/app/staticfiles/;
```

```
#     }
```

```
#
```

```
#     location / {
```

```
#         proxy_pass http://127.0.0.1:8000;
```

```
#         proxy_set_header Host $host;
```

```
#    proxy_set_header X-Real-IP $remote_addr;
#    }
#}

# Systemd service
# [Service]
# ExecStart=/home/ubuntu/venv/bin/gunicorn myproject.wsgi --workers 4
# WorkingDirectory=/home/ubuntu/app
```

109. Why choose Django? What is its purpose?

Answer:

- Batteries-included framework (admin, ORM, auth, forms built-in)
- Follows DRY (Don't Repeat Yourself) principle
- Rapid development with less code
- Built-in security (CSRF, XSS, SQL injection protection)
- Scalable for large applications
- Large ecosystem and community

Best for: Large web apps, admin panels, CMS, e-commerce, APIs with complex business logic.

110. Difference between Flask and Django? When to choose?

Answer:

Feature	Flask	Django	
-----	-----	-----	
Type	Micro-framework	Full-stack framework	
ORM	No (use SQLAlchemy)	Built-in ORM	

Admin	No	Built-in admin panel	
Flexibility	High	Opinionated	
Learning	Easier	Steeper	
Best for	Small apps, microservices	Large apps, rapid dev	

Choose Flask: Small services, APIs, prototypes, full control needed

Choose Django: Large apps, need admin panel, team projects, rapid delivery

111. Django ORM: select_related vs prefetch_related

Answer:

- select_related: Uses SQL JOIN for ForeignKey/OneToOne (single query)
- prefetch_related: Uses separate query + Python join for ManyToMany/reverse FK

Example:

N+1 Problem (BAD)

```
books = Book.objects.all()
```

```
for book in books:
```

```
    print(book.author.name) # 1 query per book!
```

select_related - single JOIN query (ForeignKey)

```
books = Book.objects.select_related('author').all()
```

```
for book in books:
```

```
    print(book.author.name) # no extra queries
```

prefetch_related - 2 queries total (ManyToMany)

```
authors = Author.objects.prefetch_related('books').all()
```

```
for author in authors:
```

```
    print(author.books.all()) # no extra queries
```

112. Django Signals

Answer: Signals allow decoupled applications to get notified when actions occur.
Common signals: pre_save, post_save, pre_delete, post_delete.

Example:

```
from django.db.models.signals import post_save
from django.dispatch import receiver
from .models import User, Profile

@receiver(post_save, sender=User)
def create_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)

# When to avoid signals:
# - When logic is specific to one view (put in view instead)
# - When it makes debugging harder
# - When order of execution matters
```

113. Django Request Lifecycle

Answer:

1. Request enters WSGI/ASGI server (Gunicorn/Uvicorn)
2. URL routing resolves to a view
3. Middleware processes request (top to bottom)
4. View executes (business logic)
5. ORM/data access

6. Template rendering or JSON serialization
7. Middleware processes response (bottom to top)
8. Response returned to client

=====

AWS INTERVIEW QUESTIONS WITH ANSWERS

=====

114. How do you create an AWS Lambda function?

Answer: Lambda is serverless compute. Create via Console, CLI, SAM, or CDK.
Write a handler function that receives event and context.

Example:

```
# --- lambda_function.py ---  
  
import json  
  
def lambda_handler(event, context):  
    """Lambda entry point"""  
    name = event.get('name', 'World')  
    return {  
        'statusCode': 200,  
        'body': json.dumps({'message': f'Hello {name}!'})  
    }  
  
# Deploy via CLI:  
# zip function.zip lambda_function.py
```

```
# aws lambda create-function \  
# --function-name my-function \  
# --runtime python3.11 \  
# --handler lambda_function.lambda_handler \  
# --role arn:aws:iam::123456789:role/lambda-role \  
# --zip-file fileb://function.zip
```

With FastAPI on Lambda (using Mangum):

```
from fastapi import FastAPI  
from mangum import Mangum
```

```
app = FastAPI()
```

```
@app.get("/")
```

```
def root():
```

```
    return {"message": "Lambda + FastAPI"}
```

```
handler = Mangum(app)
```

115. What is the maximum execution time for Lambda?

Answer: AWS Lambda maximum execution timeout is 15 minutes (900 seconds). Default is 3 seconds. For longer tasks, use Step Functions, ECS, or Batch.

116. How do you manage data in DynamoDB?

Answer: DynamoDB is a NoSQL key-value/document database. Data is organized into tables with primary keys (partition key + optional sort key).

Example:

```
import boto3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('Users')

# Create item
table.put_item(Item={
    'user_id': '123',
    'name': 'Krishna',
    'email': 'k@example.com',
    'age': 25
})

# Read item
response = table.get_item(Key={'user_id': '123'})
item = response.get('Item')

# Update item
table.update_item(
    Key={'user_id': '123'},
    UpdateExpression='SET age = :val',
    ExpressionAttributeValues={':val': 26}
)

# Delete item
table.delete_item(Key={'user_id': '123'})

# Query (requires partition key)
response = table.query(
```

```

        KeyConditionExpression=boto3.dynamodb.conditions.Key('user_id').eq('123')
    )

    # Scan (full table scan - expensive)
    response = table.scan(
        FilterExpression=boto3.dynamodb.conditions.Attr('age').gt(20)
    )

```

117. What is SQS and types of queues?

Answer: SQS (Simple Queue Service) is a managed message queue service for decoupling microservices.

Types:

- Standard Queue: At-least-once delivery, best-effort ordering, unlimited throughput
- FIFO Queue: Exactly-once processing, strict ordering, 300 msg/sec (3000 with batching)

Example:

```

import boto3

sqs = boto3.client('sqs')
queue_url = 'https://sqs.us-east-1.amazonaws.com/123456/my-queue'

# Send message
sqs.send_message(
    QueueUrl=queue_url,
    MessageBody={'task': "process_image", "id": 123}
)

```

```

# Receive messages
response = sqs.receive_message(
    QueueUrl=queue_url,
    MaxNumberOfMessages=10,
    WaitTimeSeconds=20 # long polling
)

for message in response.get('Messages', []):
    print(message['Body'])
    # Process message...
    # Delete after processing
    sqs.delete_message(
        QueueUrl=queue_url,
        ReceiptHandle=message['ReceiptHandle']
    )

# FIFO queue - requires MessageGroupId
sqs.send_message(
    QueueUrl=fifo_queue_url,
    MessageBody='order data',
    MessageGroupId='order-123',
    MessageDeduplicationId='unique-id-1'
)

```

118. What is CloudWatch?

Answer: CloudWatch is AWS monitoring service for logs, metrics, alarms, and dashboards. Used for monitoring Lambda, EC2, ECS, API Gateway, etc.

Example:

```
import boto3
import logging

# CloudWatch Logs (auto-captured from Lambda)
logger = logging.getLogger()
logger.setLevel(logging.INFO)

def lambda_handler(event, context):
    logger.info("Processing event") # goes to CloudWatch Logs
    logger.error("Something failed")
    return {"status": "done"}

# Custom metrics
cloudwatch = boto3.client('cloudwatch')

cloudwatch.put_metric_data(
    Namespace='MyApp',
    MetricData=[{
        'MetricName': 'ProcessedOrders',
        'Value': 1,
        'Unit': 'Count'
    }]
)

# Create alarm
cloudwatch.put_metric_alarm(
    AlarmName='HighErrorRate',
    MetricName='Errors',
    Namespace='AWS/Lambda',
    Statistic='Sum',
```

```
Period=300,  
EvaluationPeriods=1,  
Threshold=10,  
ComparisonOperator='GreaterThanThreshold'  
)
```

119. What is VPC? Public and Private Subnets?

Answer: VPC (Virtual Private Cloud) is an isolated virtual network in AWS.

- Public Subnet: Has route to Internet Gateway, resources get public IPs
- Private Subnet: No direct internet access, uses NAT Gateway for outbound

Example:

```
# VPC Structure:  
# VPC (10.0.0.0/16)  
#   └── Public Subnet (10.0.1.0/24)  
#       ├── EC2 (web server) - has public IP  
#       └── NAT Gateway  
#   └── Private Subnet (10.0.2.0/24)  
#       ├── EC2 (database) - no public IP  
#       └── Lambda functions
```

```
# Can you access network in private subnet?
```

```
# - Not directly from internet (no public IP)
```

```
# - Access via: VPN, Bastion Host, or AWS Systems Manager
```

```
# - Private resources can access internet via NAT Gateway (outbound only)
```

```
# Terraform example:
```

```
# resource "aws_subnet" "public" {
```

```
# vpc_id    = aws_vpc.main.id
# cidr_block = "10.0.1.0/24"
# map_public_ip_on_launch = true
# }
#
# resource "aws_subnet" "private" {
#   vpc_id    = aws_vpc.main.id
#   cidr_block = "10.0.2.0/24"
#   map_public_ip_on_launch = false
# }
```

120. How do you manage Lambda dependencies and deployment packages?

Answer: Package dependencies with your code in a ZIP or use Lambda Layers for shared libraries.

Example:

```
# Method 1: ZIP package
# mkdir package
# pip install requests -t package/
# cd package && zip -r ../deployment.zip .
# cd .. && zip deployment.zip lambda_function.py

# Method 2: Lambda Layers (shared across functions)
# mkdir python
# pip install pandas numpy -t python/
# zip -r layer.zip python/
# aws lambda publish-layer-version \
#   --layer-name my-dependencies \
#   --zip-file fileb://layer.zip \
```

```
# --compatible-runtimes python3.11

# Method 3: Docker container image
# FROM public.ecr.aws/lambda/python:3.11
# COPY requirements.txt .
# RUN pip install -r requirements.txt
# COPY lambda_function.py .
# CMD ["lambda_function.lambda_handler"]

# Method 4: SAM (Serverless Application Model)
# template.yaml:
# Resources:
#   MyFunction:
#     Type: AWS::Serverless::Function
#     Properties:
#       Runtime: python3.11
#       Handler: app.handler
#       CodeUri: ./src
```

121. What is DynamoDB Streams?

Answer: DynamoDB Streams captures item-level changes (insert, update, delete) in a DynamoDB table. Can trigger Lambda functions for real-time processing.

Example:

```
# Lambda triggered by DynamoDB Stream
def lambda_handler(event, context):
    for record in event['Records']:
        event_name = record['eventName'] # INSERT, MODIFY, REMOVE
```

```
if event_name == 'INSERT':  
    new_item = record['dynamodb']['NewImage']  
    print(f"New item: {new_item}")
```

```
elif event_name == 'MODIFY':  
    old_item = record['dynamodb']['OldImage']  
    new_item = record['dynamodb']['NewImage']  
    print(f"Updated: {old_item} -> {new_item}")
```

```
elif event_name == 'REMOVE':  
    old_item = record['dynamodb']['OldImage']  
    print(f"Deleted: {old_item}")
```

Use cases:

- Sync data to Elasticsearch

- Send notifications on changes

- Audit trail / event sourcing

- Cross-region replication

122. What is Amazon Timestream?

Answer: Timestream is a serverless time-series database for IoT, DevOps metrics, and analytics. Optimized for time-stamped data with automatic data lifecycle management.

Example:

```
import boto3
```

```
write_client = boto3.client('timestream-write')
```

```
query_client = boto3.client('timestream-query')
```



```
# Write data
```

```
write_client.write_records(  
    DatabaseName='mydb',  
    TableName='metrics',  
    Records=[  
        'Dimensions': [  
            {'Name': 'server', 'Value': 'web-01'},  
            {'Name': 'region', 'Value': 'us-east-1'}  
        ],  
        'MeasureName': 'cpu_usage',  
        'MeasureValue': '75.5',  
        'MeasureValueType': 'DOUBLE',  
        'Time': str(int(time.time() * 1000)),  
        'TimeUnit': 'MILLISECONDS'  
    ]  
)
```

```
# Query data
```

```
response = query_client.query(  
    QueryString="""  
        SELECT server, AVG(measure_value::double) as avg_cpu  
        FROM mydb.metrics  
        WHERE time > ago(1h)  
        GROUP BY server  
        """"  
)
```

Answer: Serverless means no server management. AWS handles scaling, patching, availability.

Example:

Typical serverless stack:

API Gateway → Lambda → DynamoDB

S3 → Lambda (trigger) → SQS → Lambda (worker)

SAM template example:

AWSTemplateFormatVersion: '2010-09-09'

Transform: AWS::Serverless-2016-10-31

#

Resources:

ApiFunction:

Type: AWS::Serverless::Function

Properties:

Handler: app.handler

Runtime: python3.11

MemorySize: 256

Timeout: 30

Events:

Api:

Type: Api

Properties:

Path: /users

Method: get

Policies:

- DynamoDBCrudPolicy:

TableName: !Ref UsersTable

#

```
# UsersTable:
#   Type: AWS::DynamoDB::Table
#   Properties:
#     TableName: Users
#     AttributeDefinitions:
#       - AttributeName: user_id
#         AttributeType: S
#     KeySchema:
#       - AttributeName: user_id
#         KeyType: HASH
```

```
# Best practices:
# - Keep functions small and focused
# - Use environment variables for config
# - Implement retries and dead-letter queues
# - Monitor with CloudWatch
# - Use layers for shared dependencies
```

124. AWS ECS/Fargate for deploying containers

Answer: ECS (Elastic Container Service) runs Docker containers. Fargate is serverless compute for ECS (no EC2 management needed).

Example:

```
# Dockerfile for FastAPI
# FROM python:3.11-slim
# WORKDIR /app
# COPY requirements.txt .
# RUN pip install -r requirements.txt
# COPY . .
```

```
# CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# Steps:
```

```
# 1. Build & push to ECR
```

```
# aws ecr get-login-password | docker login --username AWS --password-stdin  
<account>.dkr.ecr.<region>.amazonaws.com
```

```
# docker build -t my-api .
```

```
# docker tag my-api:latest <account>.dkr.ecr.<region>.amazonaws.com/my-  
api:latest
```

```
# docker push <account>.dkr.ecr.<region>.amazonaws.com/my-api:latest
```

```
# 2. Create Task Definition (JSON)
```

```
# {  
#   "family": "my-api-task",  
#   "containerDefinitions": [{  
#     "name": "api",  
#     "image": "<ecr-uri>:latest",  
#     "portMappings": [{"containerPort": 8000}],  
#     "memory": 512,  
#     "cpu": 256  
#   }],  
#   "requiresCompatibilities": ["FARGATE"],  
#   "networkMode": "awsvpc"  
# }
```

```
# 3. Create Service with ALB in front
```

```
# 4. Auto-scaling based on CPU/memory
```

125. What is S3 and how to use it with Python?

Answer: S3 (Simple Storage Service) is object storage for files, images, backups, static assets.

Example:

```
import boto3
```

```
s3 = boto3.client('s3')
```

```
# Upload file
```

```
s3.upload_file('local_file.pdf', 'my-bucket', 'uploads/file.pdf')
```

```
# Download file
```

```
s3.download_file('my-bucket', 'uploads/file.pdf', 'downloaded.pdf')
```

```
# Upload with metadata
```

```
s3.put_object(  
    Bucket='my-bucket',  
    Key='data/report.json',  
    Body=json.dumps({"data": "value"}),  
    ContentType='application/json'  
)
```

```
# Generate presigned URL (temporary access)
```

```
url = s3.generate_presigned_url(  
    'get_object',  
    Params={'Bucket': 'my-bucket', 'Key': 'uploads/file.pdf'},  
    ExpiresIn=3600 # 1 hour  
)
```

```
# List objects
```

```
response = s3.list_objects_v2(Bucket='my-bucket', Prefix='uploads/')
```

```
for obj in response.get('Contents', []):  
    print(obj['Key'], obj['Size'])
```